

СОДЕРЖАНИЕ

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ	5
ВВЕДЕНИЕ	6
1 Исследовательская часть	8
1.1 Задача глобальной оптимизации и метаэвристики	8
1.2 Алгоритм биогеографической оптимизации	11
1.3 Модификации модели миграции	13
1.4 Модификации базового алгоритма	14
1.5 Метод роя частиц и сравнение с биогеографической оптимизацией	16
1.6 Методы визуализации результатов оптимизации	17
1.7 Обзор программ-аналогов	19
2 Конструкторская часть	21
2.1 Функциональные требования к программному комплексу . . .	21
2.2 Архитектура программного комплекса	22
2.3 Проектирование протокола обмена данными	23
2.4 Модель хранения данных	25
2.5 Проектирование пользовательского интерфейса	26
3 Технологическая часть	28
3.1 Обоснование выбора технологий	28
3.2 Реализация вычислительного ядра сервера	29
3.3 Реализация очереди задач и обработчика	30
3.4 Реализация алгоритмов оптимизации	32
3.5 Реализация мобильного клиента	35
3.5.1 Клиент протокола	35
3.5.2 Модели данных и конверт сообщения	37
3.5.3 Сборка конфигурации задачи	37
3.5.4 Управление состоянием	38
3.5.5 Локальное хранение истории	38

3.6	Пользовательский интерфейс приложения	39
3.6.1	Подключение к серверу	39
3.6.2	Настройка численного эксперимента	40
3.6.3	Выполнение и наблюдение за прогрессом	41
3.6.4	Просмотр результата	41
3.6.5	Трёхмерная визуализация	43
3.6.6	История и сравнение запусков	44
3.7	Реализация визуализации результатов	45
3.7.1	График сходимости	45
3.7.2	Контурная карта	46
3.7.3	Трёхмерная поверхность	46
3.7.4	Анимация процесса оптимизации	47
4	Аналитическая часть	49
4.1	Тестирование программного обеспечения	49
4.1.1	Автоматическое тестирование серверной части	49
4.1.2	Функциональное тестирование клиента	50
4.2	Вычислительные эксперименты и анализ результатов	51
4.2.1	Методика экспериментов	51
4.2.2	Сравнение методов на функции с множеством мини- мумов	51
4.2.3	Сравнение методов на разных функциях	53
4.2.4	Выводы по результатам	53
	ЗАКЛЮЧЕНИЕ	54
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	55
	ПРИЛОЖЕНИЯ	58

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

В настоящей ВКР применяют следующие сокращения и обозначения.

BBO	— алгоритм биогеографической оптимизации (Biogeography-Based Optimization)
GIF	— формат обмена графическими данными (Graphics Interchange Format)
HSI	— индекс пригодности среды обитания (Habitat Suitability Index)
HTTP	— протокол передачи гипертекста (Hypertext Transfer Protocol)
JSON	— текстовый формат обмена данными (JavaScript Object Notation)
PSO	— метод роя частиц (Particle Swarm Optimization)
TCP	— протокол управления передачей (Transmission Control Protocol)
WebSocket	— протокол полнодуплексной связи поверх TCP

ВВЕДЕНИЕ

Многие практические задачи сводятся к поиску глобального минимума целевой функции. Часто у такой функции есть и множество локальных минимумов. Градиентные методы здесь малоэффективны: они сходятся к ближайшему локальному минимуму. Поэтому применяют метаэвристики — алгоритмы, работающие сразу с целой популяцией решений [1]. Один из них — алгоритм биогеографической оптимизации.

Поведение метаэвристик трудно предсказать аналитически, поэтому их оценивают по результатам запусков. Здесь помогает визуализация. По ней видно, как убывает лучшее значение, как распределяется популяция и не наступает ли преждевременная сходимость. Удобно, когда такой инструмент работает прямо на смартфоне и не требует настройки окружения. Этим обусловлена актуальность работы.

Объект исследования — модификации алгоритма биогеографической оптимизации и метод роя частиц, рассматриваемый для сравнения. Программа рассчитана на студентов, изучающих методы оптимизации, и на исследователей, которым нужно быстро сравнить метаэвристики на тестовых задачах.

Цель работы — разработка программно-аппаратного комплекса поддержки мобильного приложения визуализации результатов работы модификаций эволюционного алгоритма оптимизации.

Вычисления должны выполняться на сервере. Он поддерживает семейство методов: классический ВВО, его варианты с синусоидальной и смешанной миграцией, их модификации с адаптивной мутацией, рестартом и локальным поиском, а также метод роя частиц. Алгоритмы запускаются как на встроенных тестовых функциях, так и на функции, заданной пользователем.

Для связи смартфона с сервером нужен протокол обмена. Он поддерживает постановку задачи, передачу прогресса в реальном времени, выдачу результата, отмену и работу очереди при нескольких клиентах.

Пользователь работает с мобильным приложением. Через него настраивается эксперимент, отправляется задача и проводится наблюдение за вычислением. Главное в приложении — показать, как работает алгоритм. Для этого требуется график сходимости в реальном времени, контурная карта, трёхмерная поверхность и анимация популяции.

Комплекс нужен для сравнения методов, поэтому история запусков должна сохраняться. Сохранённые результаты сопоставляются на совмещённых графиках и в таблице. Наконец, комплекс применяется по назначению. На нём проводятся вычислительные эксперименты: методы сравниваются на функциях с различными рельефами и размерностью.

1 Исследовательская часть

1.1 Задача глобальной оптимизации и метаэвристики

Задача глобальной оптимизации состоит в поиске точки, в которой целевая функция принимает наименьшее значение:

$$f(x) \rightarrow \min, \quad x \in D \subset \mathbb{R}^d, \quad (1)$$

где f — целевая функция, D — допустимая область поиска, d — размерность пространства решений. Задача максимизации сводится к минимизации заменой f на $-f$, поэтому далее везде речь идёт о минимизации.

Сложность задачи определяется рельефом целевой функции. Если у функции единственный минимум, найти его несложно. Гораздо труднее функции с множеством локальных минимумов: наряду с глобальным минимумом у них есть много локальных, и отличить один от другого по поведению функции в окрестности точки невозможно. Классические градиентные методы движутся в сторону убывания функции и поэтому сходятся к ближайшему минимуму, не различая, локальный он или глобальный. Для таких задач они малопригодны. К тому же градиентные методы требуют, чтобы функция была дифференцируемой, а это условие выполняется не всегда.

Альтернативой служат метаэвристические алгоритмы. Это итеративные методы поиска, идеи которых заимствованы из природных процессов: эволюции, поведения стай, миграции видов. Их главная особенность — работа не с одной точкой, а с целой популяцией решений. За счёт этого алгоритм одновременно просматривает несколько областей пространства поиска, и вероятность застрять в локальном минимуме снижается. Разнообразие популяции поддерживается случайными механизмами: мутацией, обменом информацией между решениями, отбором. Метаэвристики не требуют дифференцируемости функции и не используют производные, поэтому подходят для широкого круга задач — в том числе для таких, где функция задана лишь алгоритмом её вычисления. Платят за это отсутствием гарантий: найденное решение может оказаться не глобальным минимумом, а лишь близким к нему. К этому классу относятся, в частности, генетические алгоритмы [2], метод роя частиц и рассматриваемый в работе алгоритм биогеографической оптимизации.

Чтобы сравнивать алгоритмы между собой, используют наборы тестовых функций с заранее известными минимумами [3]. Функции подбирают так, чтобы их рельеф был разным и проверял разные стороны поведения метода. В работе используются четыре такие функции.

Функция сферы — самая простая, с единственным минимумом:

$$f_{\text{sph}}(x) = \sum_{i=1}^d x_i^2. \quad (2)$$

Её глобальный минимум равен нулю и достигается в начале координат. На ней удобно оценивать скорость сходимости алгоритма, поскольку локальных ловушек нет. Поверхность и контурная карта функции сферы показаны на рисунке 1.

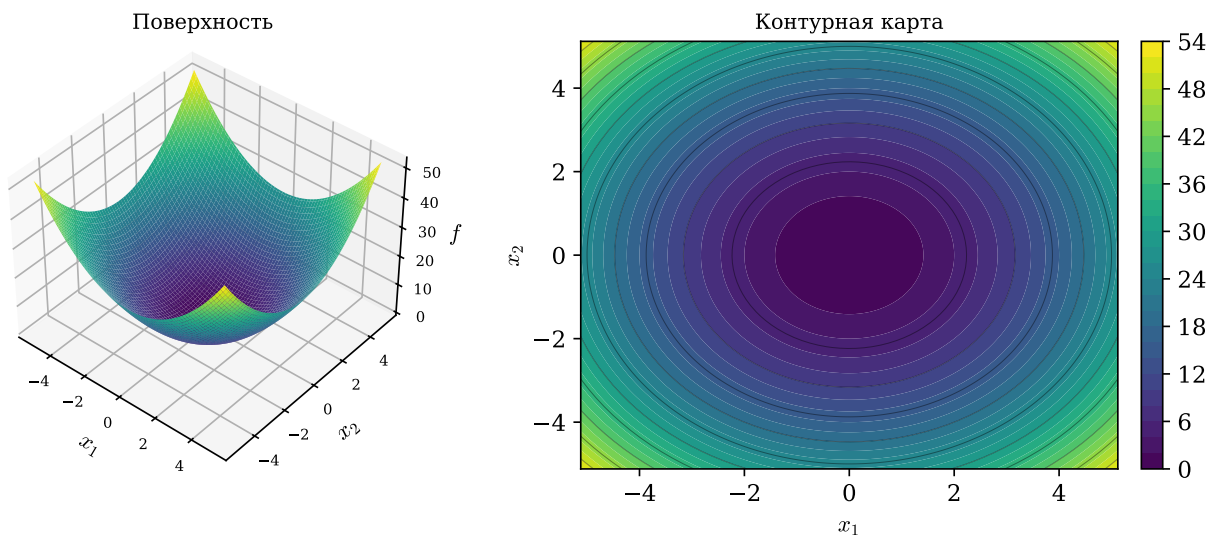


Рисунок 1 – Функция сферы: поверхность и контурная карта

У функции Растригина минимумов очень много:

$$f_{\text{ras}}(x) = Ad + \sum_{i=1}^d (x_i^2 - A \cos(2\pi x_i)), \quad A = 10. \quad (3)$$

Косинусное слагаемое создаёт большое число локальных минимумов, расположенных правильной решёткой. Глобальный минимум, равный нулю, находится в начале координат. Эта функция проверяет, способен ли алгоритм не застревать в локальных минимумах. Её рельеф приведён на рисунке 2.

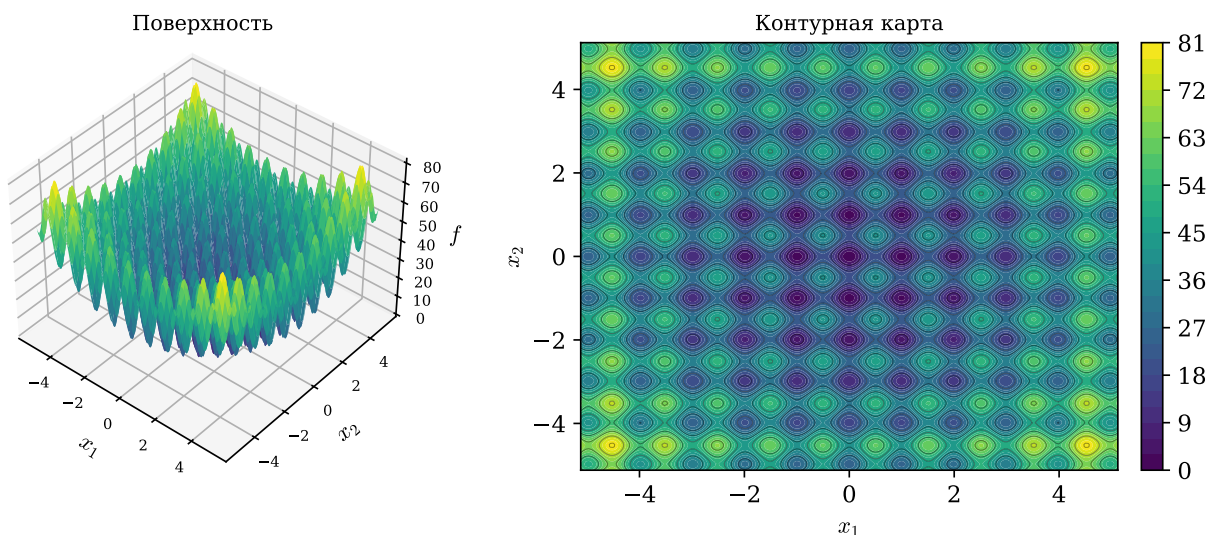


Рисунок 2 – Функция Растригина: поверхность и контурная карта

У функции Экли один глобальный минимум, окружённый почти плоской областью с мелкими неровностями:

$$f_{\text{ack}}(x) = -a \exp \left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(c x_i) \right) + a + e, \quad (4)$$

где $a = 20$, $b = 0,2$, $c = 2\pi$. Минимум, равный нулю, достигается в начале координат. Вдали от минимума функция почти плоская и плохо подсказывает направление спуска, поэтому она проверяет, насколько точно метод выходит к минимуму. Поверхность и контурная карта функции Экли показаны на рисунке 3.

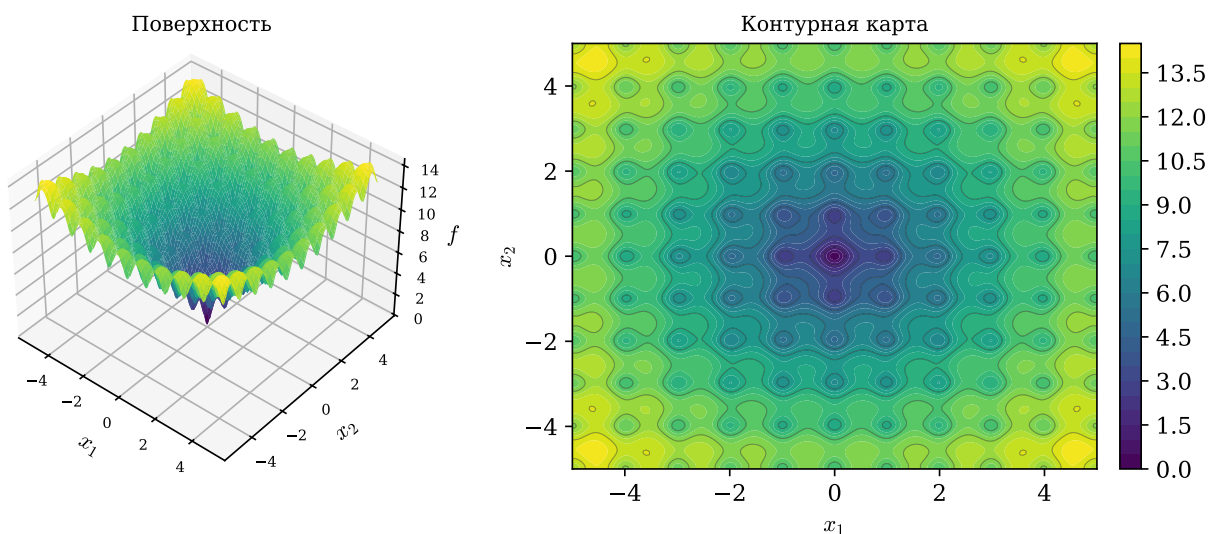


Рисунок 3 – Функция Экли: поверхность и контурная карта

Функция Букина N.6 определена только для двух переменных:

$$f_{\text{buk}}(x_1, x_2) = 100\sqrt{|x_2 - 0,01 x_1^2|} + 0,01 |x_1 + 10|. \quad (5)$$

Её минимум лежит вдоль узкого изогнутого оврага: малый шаг в сторону от него резко увеличивает значение функции. Эта функция проверяет, насколько хорошо алгоритм движется вдоль сложных по форме областей пространства решений. Её рельеф показан на рисунке 4.

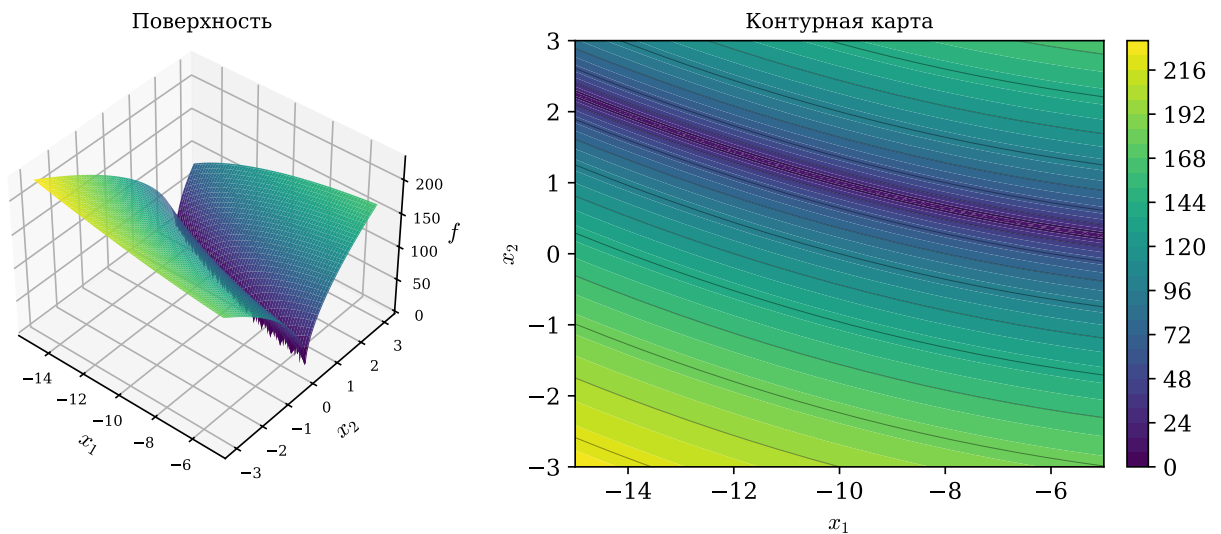


Рисунок 4 – Функция Букина N.6: поверхность и контурная карта

Такой набор охватывает задачи разного характера — от простой функции с единственным минимумом до функции с множеством локальных минимумов и до функции с узким оврагом, — что позволяет всесторонне оценить поведение метода.

1.2 Алгоритм биогеографической оптимизации

Алгоритм биогеографической оптимизации предложил Д. Саймон в 2008 году [4]. В его основе лежит модель биогеографии — науки о распределении живых организмов по островам. В терминах оптимизации каждое решение-кандидат считается отдельным островом, а качество решения — индексом пригодности среды обитания. Хорошему решению отвечает остров с высоким HSI, плохому — с низким. Хорошие острова охотно отдают свои признаки соседям, а плохие принимают чужие; так в популяции происходит обмен информацией.

У алгоритма два основных механизма — миграция и мутация. Пусть популяция состоит из N решений $X_1, \dots, X_N$, отсортированных по возрастанию значения целевой функции. Тогда ранг $r = 1$ получает худшее решение, а ранг $r = N$ — лучшее. Каждому решению сопоставляются две величины: скорость эмиграции λ_r , то есть готовность отдавать свои признаки, и скорость иммиграции μ_r — готовность принимать чужие. В линейной модели Саймона они задаются так:

$$\lambda_r = \frac{r}{N+1}, \quad \mu_r = 1 - \lambda_r, \quad r = 1, \dots, N. \quad (6)$$

Смысл формул простой: чем выше ранг решения, тем больше его скорость эмиграции и меньше скорость иммиграции. Лучшие решения становятся донорами признаков, худшие активнее принимают чужие. Вид линейных кривых миграции показан на левом графике рисунка 5.

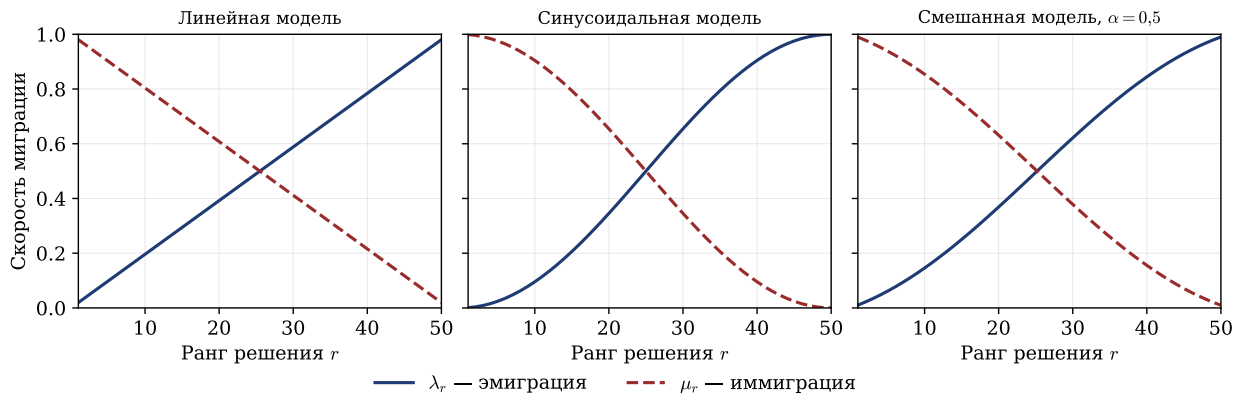


Рисунок 5 – Скорости эмиграции и иммиграции для линейной, синусоидальной и смешанной моделей миграции

Сам обмен идёт по схеме частичной иммиграции. Для каждого решения-получателя X_k , не входящего в число элитных, перебираются все его компоненты. Компонента с номером s заменяется с вероятностью μ_k : если замена происходит, на её место подставляется компонента s решения-донора. Донор выбирается случайно, причём вероятность стать донором пропорциональна скорости эмиграции λ_r , поэтому чаще донорами оказываются хорошие решения. Компоненты обновляются независимо друг от друга, и новое решение собирается из фрагментов нескольких разных доноров.

Мутация добавляет в популяцию случайность. Каждая компонента решения с вероятностью p_{mut} заменяется новым значением, равномерно выбран-

ным в допустимых границах. Это поддерживает разнообразие популяции и мешает ей преждевременно сойтись к одному минимуму.

Чтобы лучшие найденные решения не испортились при миграции и мутации, применяют элитарность: e лучших решений поколения переносятся в следующее без изменений. Один из распространённых способов сделать это таков: новые решения, полученные после миграции и мутации, оцениваются целевой функцией, после чего e худших из них заменяются сохранёнными элитными решениями.

Таким образом, одно поколение классического ВВО состоит из следующих шагов: вычислить целевую функцию для всей популяции и отсортировать решения по качеству; по рангу каждого решения найти λ_r и μ_r ; для всех неэлитных решений выполнить частичную иммиграцию, заменяя отдельные компоненты значениями доноров; применить мутацию; перенести элитные решения в новую популяцию; обновить лучшее найденное значение и историю сходимости. Эти шаги повторяются заданное число поколений.

1.3 Модификации модели миграции

В классическом ВВО скорости миграции линейно зависят от ранга решения. Форма кривой миграции определяет, насколько интенсивно лучшие решения передают свои признаки худшим и как обмен информацией распределён по популяции. Линейная модель распределяет эту интенсивность равномерно, и в ряде задач это приводит к преждевременной сходимости: решения быстро становятся похожими, а поиск теряет разнообразие [5]. Изменить поведение алгоритма можно, не трогая остальные его части, — достаточно заменить способ вычисления скоростей λ_r и μ_r .

Первый вариант — синусоидальная модель миграции. В ней кривые задаются так:

$$\lambda_r^{\sin} = \frac{1 - \cos\left(\pi \frac{r}{N}\right)}{2}, \quad \mu_r^{\sin} = 1 - \lambda_r^{\sin}, \quad r = 1, \dots, N. \quad (7)$$

Синусоидальная кривая нелинейна: на краях ранговой шкалы она пологая, а в середине меняется резко. Это видно на среднем графике рисунка 5. У группы лучших решений скорости эмиграции близки и велики, у группы худших так же близки и велики скорости иммиграции, а вблизи середины роль решения

быстро меняется с получателя на донора. В результате синусоидальная модель сильнее, чем линейная, разделяет популяцию на доноров и получателей признаков.

Второй вариант — смешанная модель. Её скорости строятся как взвешенная сумма линейной и синусоидальной кривых:

$$\lambda_r^{\text{mix}} = \alpha \lambda_r^{\text{lin}} + (1 - \alpha) \lambda_r^{\text{sin}}, \quad \mu_r^{\text{mix}} = 1 - \lambda_r^{\text{mix}}, \quad r = 1, \dots, N, \quad (8)$$

где $\alpha \in [0, 1]$ — параметр смешения, задающий долю линейной кривой. При $\alpha = 1$ модель совпадает с линейной, при $\alpha = 0$ — с синусоидальной, а промежуточные значения дают форму кривой между ними. Это позволяет настраивать баланс между широким исследованием пространства и уточнением уже найденных решений. Смешанная кривая при $\alpha = 0,5$ показана на правом графике рисунка 5.

Все три модели — линейная, синусоидальная и смешанная — используют одну и ту же схему иммиграции и мутации. Различаются они только формулами для λ_r и μ_r . Благодаря этому модель миграции остаётся самостоятельным и легко заменяемым компонентом алгоритма: переход от одного варианта к другому сводится к замене способа расчёта скоростей, а вся остальная схема поиска не меняется.

1.4 Модификации базового алгоритма

Кроме изменения формы кривых миграции, поиск можно улучшить, дополнив базовый алгоритм механизмами адаптации. Все три механизма, описанные ниже, не зависят от модели миграции и применимы к любому варианту ВВО.

Первый механизм — адаптивная мутация. В базовом алгоритме вероятность мутации p_{mut} постоянна на протяжении всей работы. Однако в начале поиска высокая мутация полезна, поскольку поддерживает разнообразие популяции и помогает охватить широкую область пространства решений, а ближе к концу она лишь вносит шум и мешает уточнять найденные минимумы. Поэтому вероятность мутации делают убывающей: она линейно снижается от начального значения P_{start} до конечного P_{end} по мере смены поколений. В

поколении t её значение равно

$$p_{\text{mut}}^{(t)} = P_{\text{start}} + \frac{t}{T-1}(P_{\text{end}} - P_{\text{start}}), \quad t = 0, 1, \dots, T-1, \quad (9)$$

где T — общее число поколений. Ранние поколения работают с высокой мутацией, поздние — с пониженной.

Второй механизм — детектор стагнации и частичный рестарт. Стагнацией называют ситуацию, когда лучшее найденное значение целевой функции не улучшается несколько поколений подряд. Базовый ВВО не умеет из неё выходить. Детектор стагнации подсчитывает поколения без улучшений; как только счётчик достигает порога W , выполняется частичный рестарт: доля ρ худших решений удаляется и заменяется новыми случайными решениями из допустимой области. Одновременно текущая вероятность мутации временно увеличивается в k раз, но не выше P_{start} . После рестарта счётчик обнуляется, а линейное убывание мутации продолжается [6]. Такой механизм помогает выбраться из локального минимума, не теряя при этом уже найденные хорошие решения.

Третий механизм — локальный поиск. Популяционные алгоритмы хорошо исследуют пространство в целом, но часто недостаточно точны при уточнении минимума. Чтобы это исправить, через заданное число поколений выбирается несколько лучших решений, и каждое из них уточняется серией шагов. На каждом шаге к решению добавляется случайное гауссовское возмущение с нулевым средним и масштабом, пропорциональным ширине допустимой области. Масштаб возмущения убывает с ростом номера поколения: в начале шаги крупнее, в конце мельче. Если возмущённая точка оказывается лучше, она принимается, иначе отвергается. Локальный поиск применяют только к лучшим решениям, чтобы не тратить вычисления на заведомо неперспективные.

Три механизма дополняют друг друга: адаптивная мутация управляет уровнем случайности на всём протяжении поиска, детектор стагнации обеспечивает выход из локальных ловушек, а локальный поиск повышает точность итогового результата. Они независимы друг от друга, поэтому могут применяться как вместе, так и по отдельности.

1.5 Метод роя частиц и сравнение с биогеографической оптимизацией

Метод роя частиц предложили Дж. Кеннеди и Р. Эберхарт в 1995 году [7]. Его идея навеяна поведением стаи птиц или косяка рыб при поиске пищи. Популяция здесь — это рой частиц, и каждая частица соответствует одному решению. Состояние частицы i задаётся положением $x_i \in \mathbb{R}^d$ и скоростью $v_i \in \mathbb{R}^d$.

На каждой итерации t скорость и положение частицы пересчитываются по формулам

$$v_i^{(t+1)} = \omega v_i^{(t)} + c_1 r_1 \left(p_{i,\text{best}}^{(t)} - x_i^{(t)} \right) + c_2 r_2 \left(g_{\text{best}}^{(t)} - x_i^{(t)} \right), \quad (10)$$

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)}, \quad (11)$$

где $p_{i,\text{best}}$ — лучшее положение, в котором частица i побывала за всё время; g_{best} — лучшее положение среди всех частиц роя; ω — инерционный коэффициент; c_1 и c_2 — когнитивная и социальная составляющие; $r_1, r_2 \sim U(0,1)$ — случайные числа [8].

Инерционный коэффициент ω задаёт баланс между исследованием пространства и уточнением найденного. При большом ω частицы сохраняют направление движения и шире обследуют область поиска, при малом — сильнее притягиваются к лучшим найденным точкам. На практике ω часто линейно убывает в ходе работы — от значения около 0,9 в начале до 0,4 в конце. Чтобы частицы не разлетались слишком далеко за одну итерацию, скорость по каждой координате обычно ограничивают по модулю некоторой величиной v_{max} , а вышедшее за границы положение возвращают в допустимую область [9].

Несмотря на внешнее сходство — оба алгоритма работают с популяцией решений и обмениваются информацией внутри неё — механизмы этого обмена устроены по-разному.

В PSO каждая частица учитывает два ориентира: своё лучшее прошлое положение и глобально лучшую точку роя. На задачах с простым рельефом это даёт быструю сходимость — рой быстро стягивается к минимуму. Но на задачах с множеством локальных минимумов тот же механизм оборачивается слабостью: если глобально лучшая точка попала в окрестность локального минимума, рой собирается там, и выбраться оттуда без перезапуска трудно.

В BBO обмен идёт покомпонентно: каждая компонента решения может быть независимо заимствована у любого донора. Это создаёт более разнообразный поток информации и мешает популяции полностью унифицироваться. Вдобавок элитарность защищает лучшие решения, а мутация постоянно вносит случайные возмущения. В сумме BBO устойчивее на сложных задачах высокой размерности, хотя на простых функциях сходится несколько медленнее [10].

Поэтому PSO и BBO занимают разные ниши. PSO удобен там, где рельеф простой и важна скорость сходимости. BBO и его модификации выигрывают на задачах с множеством локальных минимумов, где важно сохранять разнообразие популяции на протяжении всего поиска. Это согласуется с общим положением о том, что эффективность метода оптимизации зависит от свойств целевой функции и ни один метод не превосходит остальные на всех задачах [11]. В работе PSO используется как метод сравнения, относительно которого оценивается поведение вариантов BBO.

1.6 Методы визуализации результатов оптимизации

Поведение метаэвристик трудно оценить по одним лишь числам, поэтому результаты их работы принято представлять наглядно. Визуализация решает несколько задач: она позволяет наблюдать за ходом поиска и замечать нежелательные явления — преждевременную сходимость, стагнацию, потерю разнообразия популяции, — а также сравнивать алгоритмы между собой и понимать, как метод исследует пространство решений. Сложились несколько типов визуального представления, каждый из которых решает свою задачу.

Наиболее универсальное средство — график сходимости. По оси абсцисс на нём откладывается номер поколения, по оси ординат — лучшее значение целевой функции, найденное к этому поколению:

$$f_{\text{best}}^{(t)} = \min_{0 \leq \tau \leq t} f\left(x_{\text{best}}^{(\tau)}\right). \quad (12)$$

По построению такой график монотонно невозрастающий: лучшее найденное значение не может ухудшиться. По форме кривой судят о характере сходимости. Резкий начальный спад с выходом на плато говорит о быстрой сходимости к некоторому значению, которое может оказаться локальным минимумом. Плавный равномерный спад свидетельствует о планомерном ис-

следовании пространства. Ступенчатая кривая характерна для алгоритмов с рестартом: после каждого рестарта метод находит лучшее решение, и кривая делает скачок вниз. Когда значения целевой функции убывают на несколько порядков, по оси ординат применяют логарифмическую шкалу — иначе весь интересный участок кривой сжимается у нуля и становится нечитаемым. Для сравнения методов кривые их сходимости совмещают на одних осях, различая цветом и типом линии.

График сходимости показывает поведение алгоритма в терминах значений функции, но не говорит, где именно в пространстве решений находится популяция. Это показывает визуализация пространства поиска. При размерности $d = 2$ строится контурная карта: по осям откладываются координаты x_1 и x_2 , а значение $f(x_1, x_2)$ кодируется цветом, причём тёмные тона отвечают малым значениям функции, то есть перспективным областям [12]. Допустимая область разбивается на равномерную сетку, для каждой узловой точки вычисляется значение функции, и полученная матрица отображается как изображение; поверх него наносятся маркеры особей популяции. Если сетка содержит $M \times M$ точек, для построения карты требуется M^2 вычислений функции, поэтому карту рассчитывают один раз до запуска, а в ходе работы обновляют только маркеры. При $d > 2$ прямое отображение невозможно: тогда используют первые две координаты особей, а остальные фиксируют, строя сечение функции. Такой подход даёт лишь приближённое представление, но позволяет судить о расположении популяции и в многомерных задачах.

Контурная карта кодирует значение функции цветом, но не передаёт объёмной структуры рельефа: глубину минимумов и крутизну склонов по оттенку оценить трудно. Эту структуру передаёт трёхмерный поверхностный график, где значение $f(x_1, x_2)$ отображается высотой точки. Поверхность строится по той же сетке, что и контурная карта, а соседние узлы образуют четырёхугольные грани. Чтобы показать трёхмерную сцену на плоском экране, применяют проекцию, а грани рисуют по алгоритму художника — от дальних к ближним, так что ближние перекрывают дальние без использования буфера глубины.

Главное преимущество трёхмерного графика — возможность вращения. Точка обзора задаётся двумя углами: поворотом вокруг вертикальной оси φ_z и вокруг горизонтальной оси φ_x . При изменении углов координаты

каждой вершины (x, y, z) пересчитываются последовательным поворотом в двух плоскостях:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \varphi_z & -\sin \varphi_z \\ \sin \varphi_z & \cos \varphi_z \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}, \quad (13)$$

$$\begin{pmatrix} y'' \\ z'' \end{pmatrix} = \begin{pmatrix} \cos \varphi_x & -\sin \varphi_x \\ \sin \varphi_x & \cos \varphi_x \end{pmatrix} \begin{pmatrix} y' \\ z \end{pmatrix}. \quad (14)$$

Экранными координатами вершины служат x' и y'' с учётом масштаба и смещения к центру холста. Вращение позволяет рассмотреть поверхность под любым углом и заметить узкие овраги целевой функции, плохо различимые на плоской карте. Поверх поверхности можно нанести траекторию лучшего решения — последовательность точек $(x_1^{(t)}, x_2^{(t)}, f_{\text{best}}^{(t)})$, соединённых линией, которая показывает, как метод продвигался по рельефу.

Контурная карта и поверхность дают снимок одного состояния, но для понимания поиска важна динамика — как популяция смещалась от поколения к поколению. Зафиксировать её позволяет анимация в формате GIF, которая хранит последовательность кадров, сменяющих друг друга с заданной частотой [13]. Каждый кадр — это контурная карта с маркерами особей на соответствующем поколении; накопленные кадры собираются в один файл. GIF удобен тем, что не требует внешних плееров, просто собирается из готовых изображений и легко сохраняется. Его ограничение — не более 256 цветов в кадре и заметный размер файла при большом числе кадров, но для задач визуализации оптимизации это не критично.

1.7 Обзор программ-аналогов

Задача наблюдения за работой алгоритмов оптимизации не нова, и для неё уже создан ряд программных средств.

Инструмент STNWeb — это веб-приложение для анализа поведения метаэвристик. Он строит так называемые сети траекторий поиска: ориентированные графы, по которым видно, как алгоритм перемещается по пространству решений. STNWeb позволяет сравнивать несколько прогонов нескольких алгоритмов на одной задаче и помогает понять, почему тот или иной метод работает хуже [14]. Сильная сторона инструмента — глубокий анализ траекторий и удобное сравнение алгоритмов. Однако STNWeb ориентирован на

исследователей, требует подготовки входных данных в специальном формате и не показывает ход поиска в реальном времени.

Проект OptimViz решает другую задачу — он наглядно показывает, как различные методы оптимизации шаг за шагом движутся по поверхности тестовой функции, и сохраняет результат в виде анимаций [15]. Это удобно для обучения и демонстраций. Слабые стороны: инструмент написан на языке MATLAB и требует платных дополнительных пакетов, а сами анимации создаются заранее, без возможности интерактивно менять параметры запуска.

Веб-демонстрация Э. Дюпона позволяет прямо в браузере выбрать точку старта на контурной карте двумерной функции и посмотреть, как к минимуму сходятся разные алгоритмы [16]. Она интерактивна и наглядна, но ограничена двумя измерениями и фиксированным набором функций, а сами алгоритмы относятся к градиентным методам обучения нейронных сетей, а не к метаэвристикам.

Дашборд moPLOT строит детальные изображения рельефа тестовых функций, в том числе для задач многокритериальной оптимизации [17]. Он даёт качественную визуализацию ландшафта, но предназначен в первую очередь для анализа самих функций, а не для наблюдения за работой конкретного алгоритма, и требует навыков работы со средой R.

Сравнение показывает, что существующие инструменты решают близкие задачи, но каждый со своими ограничениями. Одни анализируют только завершённые запуски, другие работают лишь с двумя измерениями или с заранее подготовленными анимациями, третьи требуют специальной среды и рассчитаны на опытных исследователей. Кроме того, все рассмотренные средства предназначены для настольных компьютеров или браузера и не приспособлены для работы на мобильном устройстве.

Разрабатываемая система занимает свободную нишу. Она сочетает наблюдение за ходом поиска в реальном времени, визуализацию пространства решений в виде контурной карты и трёхмерной поверхности, анимацию динамики популяции и сравнение нескольких запусков, причём всё это доступно на мобильном устройстве и не требует от пользователя настройки вычислительного окружения. Основное внимание уделено семейству алгоритмов биогеографической оптимизации и их модификациям, что делает систему удобным средством для изучения именно этих методов.

2 Конструкторская часть

2.1 Функциональные требования к программному комплексу

Сначала нужно определить, что комплекс должен делать. Требования делятся на функциональные и нефункциональные.

Основной сценарий — проведение вычислительного эксперимента. Пользователь выбирает метод оптимизации, задаёт целевую функцию и параметры задачи и запускает вычисление. Целевой функцией может быть встроенная тестовая функция или произвольное выражение, введённое пользователем. Для задачи задаются размерность и границы области поиска, для метода — его параметры: размер популяции, число поколений, вероятность мутации.

Во время вычисления система показывает ход поиска, не дожидаясь его конца. Видны номер текущего поколения, лучшее найденное значение и график сходимости, который обновляется по мере поступления данных. Задачу можно отменить. Если она ещё не запущена, видно её место в очереди.

После завершения система представляет результат: лучшее решение, значение функции в нём, график сходимости и числовые показатели качества. Пространство поиска отображается визуализацией: контурной картой с нанесёнными решениями, трёхмерной поверхностью с возможностью вращения и анимацией популяции.

Визуализация задаёт ряд требований. График сходимости обновляется инкрементально: новые точки добавляются к уже построенной кривой, а не строятся заново. Так пользователь видит ход поиска во время вычислений, и снижается нагрузка на устройство. Экран смартфона мал, поэтому подписи осей и легенда должны быть читаемыми и не перекрывать график. Контурная карта и поверхность требуют многих вычислений функции, поэтому считаются один раз. При наблюдении и вращении меняются только маркеры и точка обзора, но не сам рельеф.

Результаты сохраняются, чтобы вернуться к ним позже. Система хранит историю запусков, позволяет просматривать её, отбирать отдельные запуски и сравнивать их — на совмещённом графике сходимости, в таблице показателей и на столбчатой диаграмме.

Нефункциональные требования задаёт назначение системы. Основное устройство — смартфон, поэтому приложение работает на мобильной плат-

форме и остаётся отзывчивым: интерфейс не подвисает при вычислениях и построении графиков. Сами вычисления длительны и ресурсоёмки, поэтому вынесены на сервер. Сервер обслуживает несколько клиентов и упорядочивает их задачи.

2.2 Архитектура программного комплекса

Возможности комплекса распределяются между двумя частями. Одна обращена к пользователю и работает на смартфоне, другая выполняет вычисления. Совмещать их в одном приложении нецелесообразно: оптимизация на больших популяциях и высоких размерностях нагружает процессор, а смартфон ограничен по мощности и заряду. Поэтому выбрана клиент-серверная архитектура: вычисления вынесены на сервер, а приложение отвечает за управление и отображение результатов. Общая структура показана на рисунке 6.

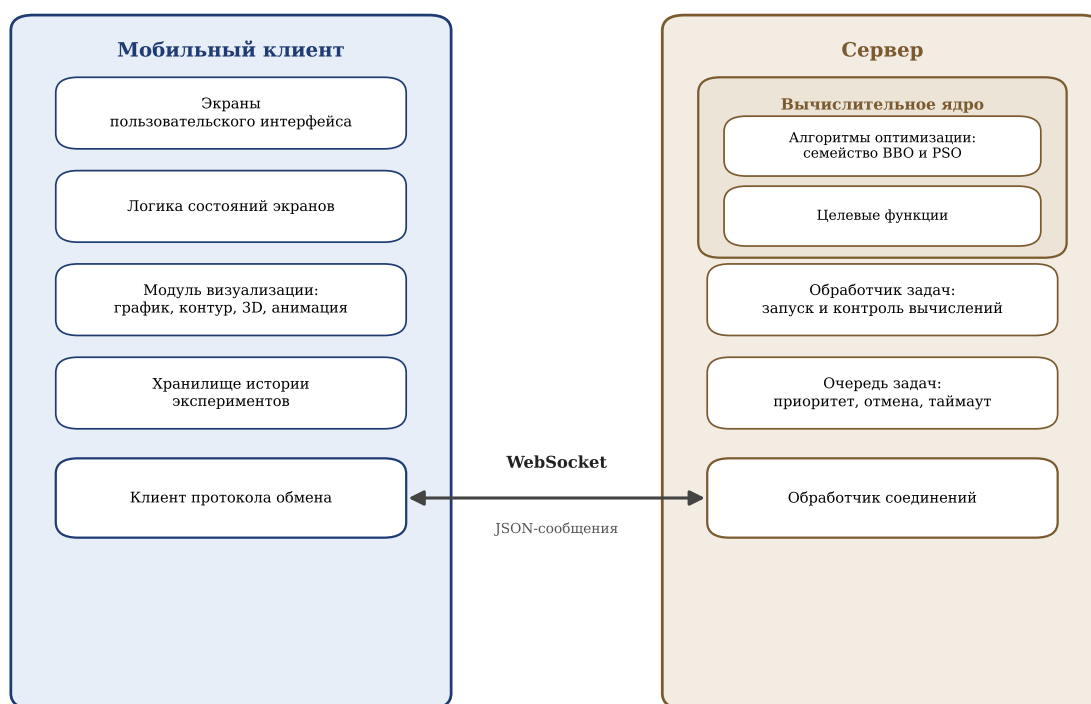


Рисунок 6 – Архитектура программного комплекса

Мобильный клиент — это приложение, с которым работает пользователь. Оно состоит из нескольких слоёв. Экраны образуют интерфейс: настройку эксперимента, наблюдение за прогрессом, просмотр результата и работу с историей. За экранами стоит слой логики состояний. Он хранит состояние каждого экрана и реагирует на действия пользователя и данные с сервера.

Отдельный модуль отвечает за визуализацию: график сходимости, контурную карту, поверхность и анимацию. История хранится в локальной базе данных на устройстве, поэтому с прошлыми результатами можно работать и без подключения. Связь с сервером обеспечивает клиент протокола.

Сервер выполняет вычисления и не имеет интерфейса. Запросы клиентов принимает обработчик соединений: он разбирает сообщения и направляет их дальше. Клиентов может быть несколько, а вычисления длительны, поэтому задачи помещаются в очередь. Она упорядочивает их и позволяет отменять ещё не выполненные. Выполнением занимается обработчик задач: он берёт задачу из очереди, запускает алгоритм и шлёт клиенту данные о прогрессе. Сами алгоритмы — семейство ВВО и метод роя частиц — вместе с целевыми функциями образуют вычислительное ядро. Какой алгоритм и функцию использовать, обработчик определяет по запросу.

Клиент и сервер связаны протоколом WebSocket. В отличие от обычного запроса и ответа по HTTP, WebSocket держит постоянное двустороннее соединение, по которому сервер сам присылает данные [18]. Это и нужно для прогресса: во время вычисления сервер шлёт сведения о каждом поколении, а приложение обновляет график в реальном времени. Сообщения передаются в формате JSON, его структура описана далее.

2.3 Проектирование протокола обмена данными

Клиент и сервер обмениваются сообщениями по постоянному соединению WebSocket. Чтобы обмен был упорядоченным и расширяемым, нужен прикладной протокол. Он задаёт, какие сообщения бывают, как они устроены и в каком порядке идут. Все сообщения передаются в формате JSON [19]. Этот формат удобочитаем, поддерживается без дополнительных библиотек и разбирается стандартными средствами обоих языков.

Каждое сообщение состоит из двух частей — заголовка и полезной нагрузки. Заголовок содержит служебные поля, общие для всех сообщений: версию протокола, тип, уникальный идентификатор, метку времени и поля для связи сообщений и идентификации задачи. Полезная нагрузка зависит от типа и содержит его данные: например, параметры задачи или сведения о поколении. Так служебную часть можно обрабатывать единообразно.

Все сообщения делятся на несколько групп по назначению. Служебные сообщения обслуживают само соединение:

- `hello` — рукопожатие при подключении и обмен сведениями о возможностях сторон;
- `ping` и `pong` — проверка того, что соединение живо;
- `error` — сообщение об ошибке протокола, проверки данных или выполнения.

Сообщения управления задачами обеспечивают её постановку и сопровождение:

- `job.submit` — запрос на запуск задачи с указанием метода, целевой функции и параметров;
- `job.accepted` — подтверждение приёма задачи и присвоение ей идентификатора;
- `cancel` — запрос на отмену задачи;
- `job.status.get` и `job.status` — запрос текущего состояния задачи и ответ на него.

Сообщения о ходе и итогах вычисления идут от сервера к клиенту:

- `job.started` — уведомление о начале выполнения задачи;
- `job.progress` — данные об очередном поколении: его номер и лучшее найденное значение;
- `job.result` — итоговый результат с лучшим решением и показателями качества;
- `job.finished` — уведомление о завершении задачи с указанием итогового состояния.

Порядок смены состояний задачи и отправки этих сообщений образует её жизненный цикл, показанный на рисунке 7. Поступившая задача попадает в очередь. Когда обработчик берёт её в работу, она переходит в выполнение, и клиенту уходит уведомление о начале. Во время выполнения сервер периодически шлёт прогресс. Завершиться задача может по-разному: успешно, с ошибкой, по отмене или по превышению времени. Отменить её можно и пока она стоит в очереди.

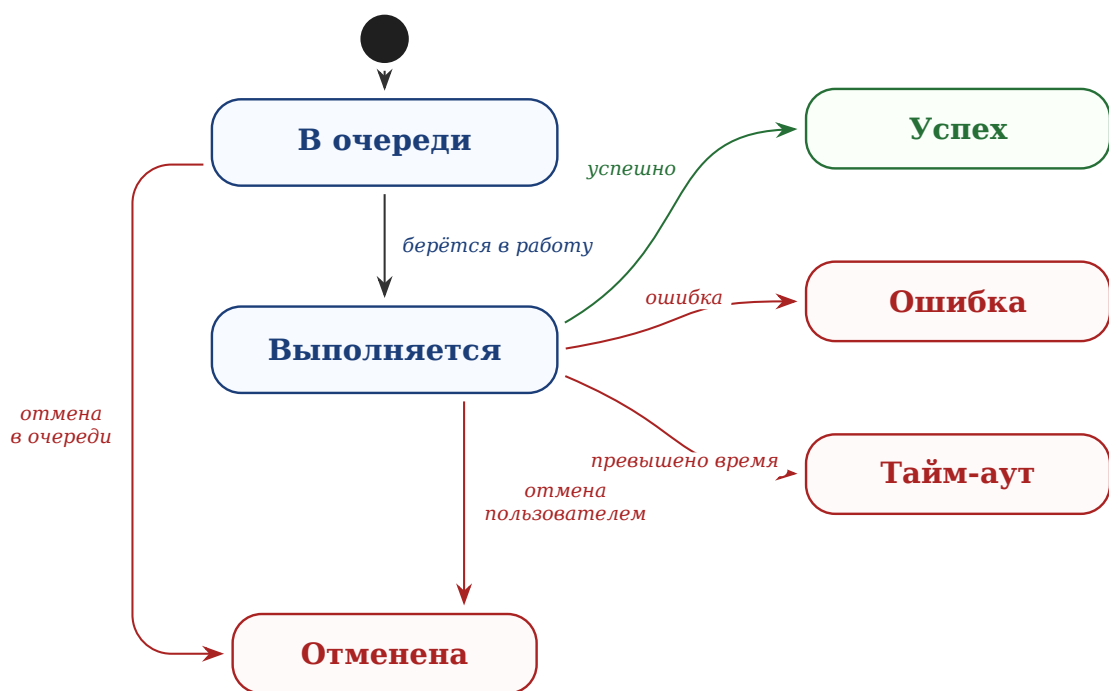


Рисунок 7 – Жизненный цикл задачи на сервере

Протокол предусматривает и передачу больших данных. Если сообщение превышает допустимый размер, его нагрузку можно разбить на части — чанки — и собрать на клиенте по идентификатору потока. Заголовок допускает и сжатие нагрузки. Эти средства заложены на будущее: при текущих объёмах они не нужны, и сообщения идут целиком в несжатом виде.

2.4 Модель хранения данных

Система должна сохранять историю экспериментов, чтобы пользователь возвращался к прошлым запускам и сравнивал их. Отсюда вытекает несколько задач хранилища. Во-первых, по каждому запуску нужно сохранять достаточно сведений, чтобы опознать его в списке и понять, что вычислялось: каким методом, на какой задаче и с каким итогом. Во-вторых, нужно хранить сам результат — не только итоговое число, но и данные для графиков и визуализации. В-третьих, записи должны быть однородными, чтобы их можно было перебирать, отбирать и сопоставлять.

Чтобы учесть эти требования, нужно понять, какие данные и в каких отношениях хранить. Для этого построена концептуальная модель данных в виде диаграммы «сущность-связь» на рисунке 8. Она описывает предметную область на уровне сущностей и связей, без привязки к способу хранения.

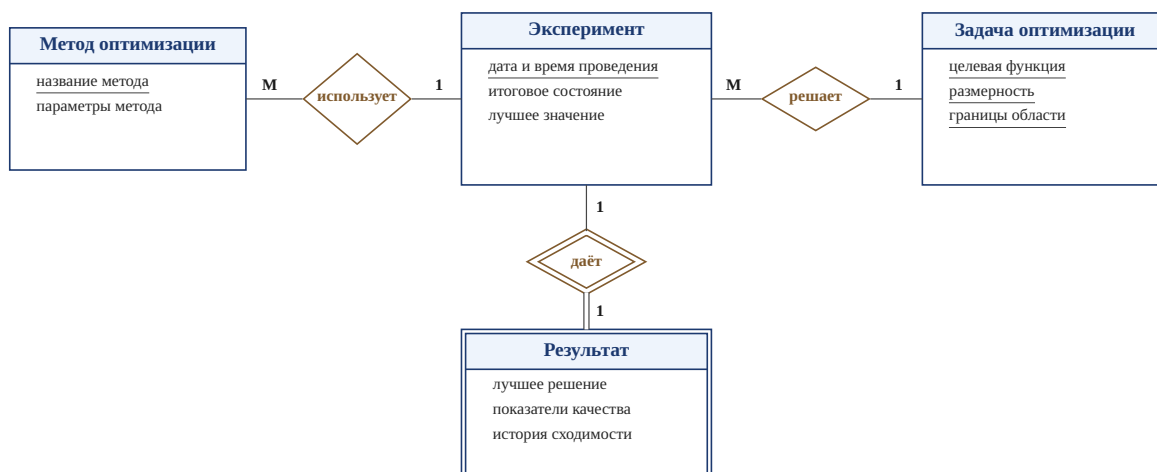


Рисунок 8 – Модель «сущность-связь» истории экспериментов

Центральная сущность — эксперимент. Он соответствует одному завершённому запуску и описывается датой и временем проведения, итоговым состоянием и лучшим найденным значением. Ключом служит дата и время: запуск определяется моментом начала.

С экспериментом связаны три сущности. Метод оптимизации хранит название метода и значения его параметров; ключом служит название. Один метод используется во многих экспериментах, поэтому связь «использует» имеет тип «многие к одному». Задача оптимизации состоит из целевой функции, размерности и границ области поиска. Эти три атрибута образуют составной ключ: одна функция в разных размерностях задаёт разные задачи. Связь «решает» тоже имеет тип «многие к одному».

Результат хранит итог запуска: лучшее решение, числовые показатели и историю сходимости. Он не имеет собственного ключа и не существует отдельно от эксперимента, поэтому моделируется как слабая сущность. Связь «даёт» идентифицирующая, тип «один к одному»: каждому эксперименту соответствует один результат. В итоге эксперимент объединяет вокруг себя метод, задачу и результат, а история — это набор таких однородных записей на устройстве пользователя.

2.5 Проектирование пользовательского интерфейса

Интерфейс строится вокруг основного сценария: настроить эксперимент, запустить его, понаблюдать за вычислением и изучить результат. Каждый шаг вынесен на отдельный экран, чтобы не перегружать пользователя.

Нужны также экраны истории и сравнения. Переходы между экранами показаны на рисунке 9.

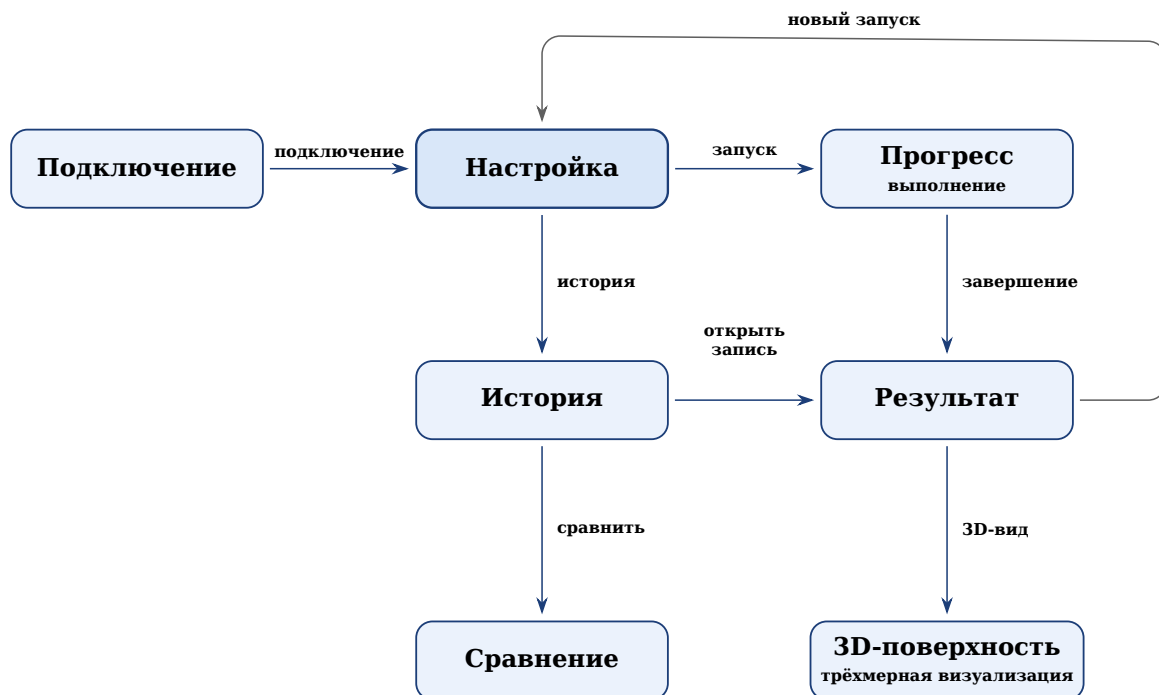


Рисунок 9 – Схема навигации между экранами приложения

Работа начинается с экрана подключения: задаётся адрес сервера и устанавливается соединение. Затем открывается экран настройки — центральный экран приложения. Здесь выбирается метод оптимизации, задаются целевая функция, размерность, границы области поиска и параметры метода. Отсюда же можно перейти к истории.

После запуска открывается экран прогресса. На нём видны номер текущего поколения, лучшее значение и график сходимости в реальном времени. По завершении приложение переходит на экран результата. Он показывает лучшее решение, числовые показатели, график сходимости за весь запуск и контурную карту пространства поиска. С него можно открыть трёхмерную поверхность, вернуться к настройке или перейти к истории. Завершённый эксперимент сохраняется в историю.

Экран истории содержит список прошлых экспериментов. Любую запись можно открыть заново, а отметив несколько — перейти к сравнению. На экране сравнения запуски выводятся вместе: на совмещённом графике, в таблице и на столбчатой диаграмме. Такое разделение даёт простой путь от постановки задачи до анализа результатов.

3 Технологическая часть

3.1 Обоснование выбора технологий

Архитектура, описанная в конструкторской части, не привязана к конкретным технологиям. В этом подразделе обосновывается выбор средств, которыми комплекс реализован: транспортного протокола, серверного и клиентского языков, модели параллельной обработки и формата обмена данными.

Транспортный протокол должен удовлетворять трём условиям. Сервер обязан слать данные клиенту сам, после каждого поколения, не дожидаясь запроса. Соединение должно оставаться открытым всё время работы задачи. Накладные расходы на сообщение должны быть малыми: прогресс идёт потоком, по одному сообщению на поколение. Не все решения отвечают этим условиям. Опрос сервера по HTTP прост, но неэффективен: большинство запросов возвращают пустой ответ, а интервал опроса задаёт задержку данных [20]. Однонаправленная потоковая передача событий снимает проблему опроса, но не даёт обратной связи: клиент не может по тому же соединению отправить задачу или отмену. Всем условиям отвечает WebSocket. Он держит постоянное двустороннее соединение поверх TCP, обе стороны шлют сообщения в любой момент, а заголовок кадра занимает лишь несколько байт [21]. Поэтому транспорт — WebSocket.

Серверная часть написана на Python. Этот язык стал стандартным в научных вычислениях во многом благодаря библиотеке NumPy [22] с её быстрыми операциями над массивами. Действия над всей популяцией — вычисление функции, сортировка, отбор доноров — выражаются через массивы NumPy и идут на уровне скомпилированного кода. Это намного быстрее циклов на чистом Python.

Сервер должен обслуживать несколько клиентов сразу, не останавливаясь на время чужих вычислений. Для этого применяется асинхронная модель `asyncio`: много соединений живут в одном потоке, и операция, ждущая данных по сети, уступает управление другим [23]. Но алгоритм оптимизации нагружен и не имеет точек ожидания. Запущенный прямо в цикле событий, он заблокировал бы остальные соединения. Поэтому алгоритм идёт в отдельном потоке из пула, а прогресс возвращается в цикл событий через потокобезопас-

ную очередь. Веб-часть построена на FastAPI: он поддерживает WebSocket и работает поверх той же модели [24].

Сообщения сериализуются в JSON. Формат поддерживается без дополнительных библиотек и в Python, и в Dart [25, 26], а текст удобно читать при отладке. Двоичные форматы компактнее, но выигрыш здесь невелик: сообщения о прогрессе содержат немного чисел, и их сериализация ничтожна на фоне вычислений [27]. Для оценки: финальная популяция 50×10 — это 500 чисел, около 5–7 КБ в JSON. По умолчанию в потоке идут лишь номер поколения и лучшее значение. Объёмные данные — популяция, пригодность, скорости — передаются только по запросу клиента.

Клиентская часть написана на Dart с фреймворком Flutter. Dart имеет строгую статическую типизацию и встроенную поддержку асинхронности [28]. Особенно полезна абстракция потока событий: сообщения по WebSocket естественно ложатся в такую последовательность, на которую приложение подписывается. Flutter — кроссплатформенный фреймворк, компилирующий приложение в нативный код со своим движком рендеринга [29]. Интерфейс описывается деревом виджетов, и при новых данных перестраивается только изменившаяся часть. Это и позволяет плавно обновлять график в реальном времени. Контурная карта и поверхность рисуются через низкоуровневый холст с доступом к графическим примитивам.

3.2 Реализация вычислительного ядра сервера

Вычислительное ядро отвечает за запуск алгоритмов. Оно состоит из трёх частей: реестра методов, набора целевых функций и связующего модуля запуска. Так новые алгоритмы и функции добавляются, не меняя остальной код.

Реестр методов — единая точка, где перечислены все алгоритмы. Каждому методу сопоставлена пара из класса алгоритма и класса конфигурации, а ключом служит имя метода из запроса клиента. Содержимое реестра показано в листинге 1.

Чтобы создать алгоритм, фабрика находит пару по имени метода, собирает конфигурацию из присланных параметров с размерностью и границами и возвращает готовый экземпляр. Если имени в реестре нет, фабрика сообщает об ошибке. Поэтому добавление алгоритма сводится к записи в реестр.

Листинг 1: Реестр методов оптимизации

```
1 METHODS = {
2     "bbo.classic": (ClassicBBO, ClassicBBOConfig),
3     "bbo.classic.modified": (ClassicBBO_Modified, BBOConfigModified),
4     "bbo.sinusoidal": (BBO_Sinusoidal, SinBBOConfig),
5     "bbo.sinusoidal.modified": (BBO_Sinusoidal_Modified,
6         BBOConfigSinModified),
7     "bbo.mixed": (BBO_Mixed, MixedBBOConfig),
8     "bbo.mixed.modified": (BBO_Mixed_Modified, BBOConfigMixedModified),
9     "pso": (PSO, PSOConfig),
10 }
```

Целевая функция создаётся отдельно. Видов два. Встроенные функции — сфера, Растригина, Экли и Букина N.6 — выбираются по имени. Кроме того, пользователь может задать функцию выражением от координат. Выполнять выражение напрямую небезопасно: в нём может быть обращение к файлам или иной вредный код. Поэтому выражение не исполняется как код, а разбирается в синтаксическое дерево и проверяется. Разрешены только арифметика и функции из ограниченного набора: корень, экспонента, логарифм, тригонометрические и подобные. Любое постороннее имя или обращение к атрибуту отклоняет выражение. Только проверенное выражение компилируется в функцию. Ей доступны лишь координаты и разрешённые функции, а встроенные средства языка закрыты.

Связующий модуль выполняет один запуск от начала до конца. Он разбирает параметры задачи, строит границы, создаёт функцию и алгоритм и запускает оптимизацию. После каждого поколения алгоритм вызывает функцию обратного вызова. Через неё модуль отдаёт прогресс наружу и проверяет, не пришла ли отмена; если пришла — прерывает работу. По завершении результат готовится к передаче: массивы переводятся в списки, добавляются время счёта и имя метода.

3.3 Реализация очереди задач и обработчика

Сервер обслуживает несколько клиентов, а каждый запуск занимает заметное время. Поэтому задачи не выполняются сразу, а проходят через очередь. За постановку и извлечение отвечает очередь задач, за выполнение — обработчик.

Очередь построена на приоритетной очереди из `asyncio`. Её элемент — тройка из приоритета, порядкового номера постановки и идентификатора

задачи. Приоритетов три: высокий, обычный и низкий. При равных приоритетах задачи идут в порядке поступления — для этого и нужен порядковый номер. Сам объект задачи лежит в отдельном реестре, а в очереди — только идентификатор.

При постановке очередь решает ещё две задачи. Первая — защита от повторов. Клиент даёт каждому запросу уникальный идентификатор; при повторном приходе очередь находит совпадение и возвращает уже созданную задачу. Вторая — защита от переполнения: число ожидающих задач ограничено, и при достижении предела постановка отклоняется. По идентификатору также можно узнать позицию в очереди и время ожидания и отменить задачу — ожидающую или выполняемую.

Выполнением занимается обработчик. При запуске сервера создаётся пул обработчиков, их число задаётся в настройках. Каждый работает в цикле: берёт задачу, переводит её в выполнение, уведомляет клиента и запускает оптимизацию. Так сервер считает несколько задач сразу. Алгоритм нагружен и не имеет точек ожидания, поэтому в асинхронном цикле его запускать нельзя — он заблокировал бы соединения. Поэтому алгоритм идёт в отдельном потоке из пула, а цикл тем временем обслуживает других клиентов.

На выполнение задаётся ограничение по времени. Запуск обёрнут в ожидание с таймаутом: если срок вышел, вычисление прерывается, а задача переходит в состояние превышения времени. Так же обрабатываются успех, ошибка и отмена: каждому исходу соответствует свой переход и своё сообщение клиенту.

Отдельно стоит передача прогресса. Алгоритм идёт в рабочем потоке, а отправка — в асинхронном цикле, поэтому напрямую данные между ними не передать. Прогресс идёт через потокобезопасную очередь: после каждого поколения рабочий поток кладёт в неё данные, а отдельная задача в цикле их извлекает и шлёт клиенту. Поддерживаются режимы: сообщать о каждом поколении, о каждом k -м или не сообщать — так снижается плотность потока. По окончании в очередь кладётся признак конца, и отправка корректно останавливается.

Связь очереди с клиентами обеспечивает обработчик соединений. Для каждого клиента создаётся свой обработчик, ведущий диалог по протоколу. Сначала он принимает приветствие и сообщает о возможностях сервера:

доступные методы, виды целевых функций, режимы прогресса и предельный размер сообщения. Если первым пришло не приветствие, соединение отклоняется. После рукопожатия обработчик читает сообщения и направляет каждое по типу: запуск ставит задачу в очередь, отмена снимает её, запрос состояния возвращает сведения, проверка связи подтверждается ответом. Сообщения неизвестного типа или с ошибкой в данных не прерывают работу — в ответ идёт сообщение об ошибке. Когда клиент отключается, обработчик не отменяет его задачи сразу, а запускает отложенную отмену со сроком в 2 минуты. Если клиент успеет переподключиться и запросить состояние, задача привязывается к новому соединению, и отмена снимается. Иначе задача отменяется, чтобы не тратить ресурсы впустую.

Перед постановкой параметры задачи проверяются. Проверка охватывает структуру сообщения и значения: имя метода должно быть известным, размерность и размер популяции — положительными, границы — согласованными. Некорректный запрос отклоняется с понятным сообщением ещё до запуска вычислений.

3.4 Реализация алгоритмов оптимизации

Все семь методов реализованы по общей схеме. Каждый алгоритм — отдельный класс, получающий при создании конфигурацию, целевую функцию и функцию обратного вызова. Основная работа идёт в методе `optimize`: он инициализирует популяцию, проводит заданное число поколений и возвращает лучшее решение, его значение, историю сходимости и финальную популяцию. Различаются методы только тем, что происходит внутри одного поколения.

Параметры метода описаны классом конфигурации. В нём заданы значения по умолчанию и проверки: размер популяции не меньше двух, вероятность мутации в отрезке от нуля до единицы, число элитных решений меньше размера популяции. Незаданные границы заполняются по умолчанию. Проверки идут при создании конфигурации, поэтому неверные параметры отсекаются до запуска. Случайность всех методов исходит из одного генератора с заданным `seed` — это делает запуски воспроизводимыми.

Рассмотрим классический ВВО. Одно поколение показано в листинге 2. Сначала популяция сортируется по значению функции, затем по рангам счи-

таются скорости эмиграции и иммиграции. Для каждого неэлитного решения идёт покомпонентная иммиграция: компонента заменяется значением донора с вероятностью, равной скорости иммиграции, а донор выбирается с вероятностью, пропорциональной скорости эмиграции. После миграции применяется мутация, а в конце поколения лучшие решения возвращаются вместо худших потомков.

Листинг 2: Одно поколение классического ВВО

```

1 order = np.argsort(fitness)
2 X, fitness = X[order], fitness[order]
3 best_f, best_x = float(fitness[0]), X[0].copy()
4
5 lambdas, mus = self._rank_based_migration_rates(self.cfg.pop_size)
6 donor_probs = lambdas[:, -1] / lambdas[:, -1].sum()
7 donor_cdf = np.cumsum(donor_probs)
8
9 Z = X.copy()
10 for k in range(self.cfg.elite_count, self.cfg.pop_size):
11     imm_flags = self.rng.random(self.cfg.dims) < mus[:, -1][k]
12     for s in np.where(imm_flags)[0]:
13         j = self._roulette_select(donor_cdf)
14         Z[k, s] = X[j, s]
15     mut_flags = self.rng.random(self.cfg.dims) < self.cfg.mutation_rate
16     Z[k, mut_flags] = self.low[mut_flags] + self.rng.random(mut_flags.
17         sum()) * self.span[mut_flags]
18
19 fit_Z = self._evaluate_population(self._clip(Z))
20 worst = np.argsort(fit_Z)[-self.cfg.elite_count:]
21 Z[worst], fit_Z[worst] = X[:self.cfg.elite_count], fitness[:self.cfg.
    elite_count]
22 X, fitness = Z, fit_Z

```

Варианты с синусоидальной и смешанной миграцией построены по тому же циклу. Отличие — только в способе вычисления скоростей λ_r и μ_r . Остальное совпадает с классическим.

Модифицированные версии устроены сложнее. К базовому циклу добавлены три механизма: адаптивная мутация, частичный рестарт и локальный поиск. Структура поколения показана в листинге 3. Сначала вычисляется текущая вероятность мутации, линейно убывающая от начального значения к конечному. Затем проверяется счётчик стагнации: если лучшее решение долго не улучшалось, идёт частичный рестарт, а вероятность мутации временно повышается. После этого — миграция, мутация, элитарность и при необхо-

димости локальный поиск. В конце счётчик стагнации обновляется по тому, улучшилось ли решение.

Листинг 3: Одно поколение модифицированного ВВО

```
1 mutation_rate = self._current_mutation_rate(gen)
2 if stagnation >= self.cfg.stagnation_generations:
3     X, fit = self._restart_partially(X, fit)
4     mutation_rate = min(self.cfg.mutation_start,
5                          mutation_rate * self.cfg.mutation_boost_factor)
6     stagnation = 0
7
8 X_new = self._migration_step(X, fit)
9 X_new = self._mutation_step(X_new, mutation_rate)
10 fit_new = self._evaluate(X_new)
11 X, fit = self._apply_elitism(X, fit, X_new, fit_new)
12
13 if self.cfg.enable_local_search and (gen + 1) % self.cfg.
14     local_search_interval == 0:
15     X, fit = self._local_search(X, fit, gen)
16
17 cur_best = float(fit[np.argmin(fit)])
18 if cur_best < best_f:
19     best_f, stagnation = cur_best, 0
20 else:
21     stagnation += 1
```

Частичный рестарт — это замена доли худших решений новыми случайными. Число заменяемых задаётся долей от размера популяции, элитные решения защищены. Локальный поиск уточняет несколько лучших решений: к каждому добавляется случайное гауссовское возмущение, масштаб которого пропорционален ширине области и убывает с номером поколения. Лучшая точка принимается, иначе отвергается. Так точность растёт вблизи найденных минимумов, не затрагивая остальную популяцию.

Метод роя частиц реализован матрично: вся популяция обновляется группой операций над массивами, без цикла по частицам. Одна итерация показана в листинге 4. Скорости пересчитываются сразу: к инерционной части добавляются когнитивная, направленная к личному лучшему, и социальная, направленная к глобально лучшему. Затем скорости ограничиваются по модулю, положения сдвигаются и отсекаются по границам, после чего обновляются личные и глобально лучшее решения.

Листинг 4: Одна итерация метода роя частиц

```
1 w = self._inertia_weight(t)
2 r1, r2 = self.rng.random((N, n)), self.rng.random((N, n))
3
4 cognitive = self.cfg.c1 * r1 * (pbest_pos - X)
5 social     = self.cfg.c2 * r2 * (gbest_pos - X)
6 V = w * V + cognitive + social
7 V = np.clip(V, -self.vmax, self.vmax)
8
9 X = np.clip(X + V, self.low, self.high)
10 fit = self._evaluate(X)
11
12 improved = fit < pbest_fit
13 pbest_pos[improved], pbest_fit[improved] = X[improved], fit[improved]
14 g_idx = int(np.argmin(pbest_fit))
15 if pbest_fit[g_idx] < gbest_fit:
16     gbest_fit, gbest_pos = float(pbest_fit[g_idx]), pbest_pos[g_idx].
        copy()
```

Использование массивов NumPy здесь принципиально. Оценка функции для всей популяции, сортировка, отбор и пересчёт скоростей идут как операции над матрицами, а не как циклы на чистом Python. Для PSO это убирает цикл по частицам полностью. В ВВО покомпонентная иммиграция всё же требует перебора, ведь донор для каждой компоненты выбирается отдельно, но оценка и мутация остаются векторными. Вид функции учитывается: сначала она вызывается для всей матрицы решений, и лишь если так не вышло — например, для поточечного пользовательского выражения, — идёт поэлементный расчёт. После каждого поколения методы вызывают функцию обратного вызова с номером поколения и лучшим значением — эти данные и уходят клиенту.

3.5 Реализация мобильного клиента

Мобильный клиент построен по слоям. Нижний отвечает за связь по протоколу, средний хранит состояние экранов и обрабатывает события, верхний отображает интерфейс. Дополнительно выделены слой моделей данных и слой хранения истории. Так одну часть можно менять, не трогая остальные.

3.5.1 Клиент протокола

Связь с сервером заключена в отдельном классе. Он скрывает детали WebSocket и даёт приложению единый поток событий. Сообщения сервера

разбираются в типизированные события и публикуются в широковещательный поток, на который подписываются нужные части приложения. Приём и маршрутизация показаны в листинге 5: строка разбирается в конверт, из него берутся тип и нагрузка, и по типу создаётся событие.

Листинг 5: Приём и маршрутизация сообщений на клиенте

```
1 void _onMessage(String raw) {
2   final envelope = parseEnvelope(raw);
3   final type = getType(envelope);
4   final payload = getPayload(envelope);
5
6   switch (type) {
7     case MessageTypes.jobAccepted:
8       _eventController.add(WsJobAccepted(JobAccepted.fromPayload(
9         payload)));
10    case MessageTypes.jobStarted:
11      _eventController.add(WsJobStarted(JobStarted.fromPayload(payload)
12        ));
13    case MessageTypes.jobProgress:
14      _eventController.add(WsJobProgress(JobProgress.fromPayload(
15        payload)));
16    case MessageTypes.jobResult:
17      _eventController.add(WsJobResult(JobResult.fromPayload(payload)))
18      ;
19    case MessageTypes.jobFinished:
20      _eventController.add(WsJobFinished(JobFinished.fromPayload(
21        payload)));
22    case MessageTypes.error:
23      _eventController.add(WsError(ProtocolError.fromPayload(payload)))
24      ;
25  }
26 }
```

Клиент устойчив к обрывам связи. При неожиданном закрытии он восстанавливает соединение сам, и задержка между попытками растёт от одной секунды до тридцати, чтобы не нагружать сеть. Пока связь есть, клиент периодически шлёт проверку. При подключении он отправляет приветствие с запросом потока прогресса. Намеренное отключение помечается особым признаком — тогда восстановление не запускается. Кроме приёма событий клиент даёт методы для отправки задачи, запроса состояния и отмены.

3.5.2 Модели данных и конверт сообщения

Сообщения разбираются и собираются в отдельном слое моделей. Каждому типу сообщения соответствует свой класс с методом разбора нагрузки, поэтому остальной код работает с типизированными объектами, а не со словарями. Исходящие сообщения строит общая функция конверта. Она добавляет служебные поля заголовка: версию протокола, тип, идентификатор сообщения, метку времени и поля для связи сообщений и задачи. Функция показана в листинге 6.

Листинг 6: Построение конверта исходящего сообщения

```
1 Map<String, dynamic> buildEnvelope({
2   required String type,
3   required Map<String, dynamic> payload,
4   String? clientReqId,
5   String? jobId,
6 }) {
7   return {
8     'header': {
9       'v': 1,
10      'type': type,
11      'msg_id': _uuid.v4(),
12      'ts': DateTime.now().toUtc().toIso8601String(),
13      'trace': {'client_req_id': clientReqId, 'job_id': jobId},
14    },
15    'payload': payload,
16  };
17 }
```

3.5.3 Сборка конфигурации задачи

Параметры эксперимента с экрана настройки собираются из нескольких частей: целевой функции, постановки задачи, метода, режима потоковой передачи и состава выходных данных. Описание функции хранит её вид — встроенная или заданная выражением — и соответствующие поля. Постановка задаёт размерность и границы области, единые для всех координат или свои для каждой. Параметры метода зависят от алгоритма: у ВВО это размер популяции и вероятность мутации, у роя частиц — коэффициенты. Каждая часть умеет переводить себя в представление для передачи, а перед отправкой части объединяются в единый запрос.

3.5.4 Управление состоянием

Средний слой построен на однонаправленном потоке данных. Каждому экрану соответствует объект состояния. Он подписан на поток событий клиента, обрабатывает их и порождает новое неизменяемое состояние, на которое реагирует интерфейс. Таких объектов несколько: подключение, сборка конфигурации, ход задачи и работа с историей. Показателен объект экрана прогресса. При создании он отправляет задачу и подписывается на события, а получая прогресс, инкрементально накапливает историю лучших значений, как в листинге 7. Новые значения дописываются в конец, и график достраивается в реальном времени, не перестраиваясь.

Листинг 7: Накопление истории прогресса

```
1 case WsJobProgress(:final data):
2   final newHistory = List<double>.from(state.historyBestF);
3   if (data.progress.historyBestFTail != null) {
4     for (final v in data.progress.historyBestFTail!) {
5       if (newHistory.isEmpty || newHistory.last != v) newHistory.add(v)
6     }
7   } else {
8     newHistory.add(data.progress.bestF);
9   }
10  emit(state.copyWith(
11    phase: JobPhase.running,
12    iteration: data.progress.iteration,
13    bestF: data.progress.bestF,
14    historyBestF: newHistory,
15  ));
```

Этот же объект следит за фазой задачи: ожидание в очереди, начало, ход вычисления и завершение. При сообщении о завершении фаза переходит в одно из конечных состояний — успех, ошибка, отмена или превышение времени, — и интерфейс меняет вид. Отдельно копится история лучших решений в пространстве: по ней строится траектория на трёхмерной визуализации.

3.5.5 Локальное хранение истории

История завершённых экспериментов хранится на устройстве в локальной базе данных. Результат запуска сериализуется в текст и сохраняется отдельной записью с метаданными: методом, функцией, размерностью, итоговым значением и временем. При открытии истории записи читаются обратно

в типизированные объекты, а метаданные позволяют отбирать и фильтровать запуски. Хранение на устройстве делает историю доступной и без подключения к серверу.

3.6 Пользовательский интерфейс приложения

В этом подразделе показано готовое приложение. Интерфейс выдержан в едином стиле: общая цветовая гамма, крупные элементы под сенсорный экран и привычные компоненты — выпадающие списки, переключатели и ползунки. Экраны идут в том порядке, в каком их проходит пользователь.

3.6.1 Подключение к серверу

С этого экрана начинается работа. Пользователь вводит адрес и порт сервера и нажимает кнопку подключения; под ней показано состояние соединения. Адрес последнего сервера сохраняется, поэтому вводить его заново не нужно. Экран показан на рисунке 10.

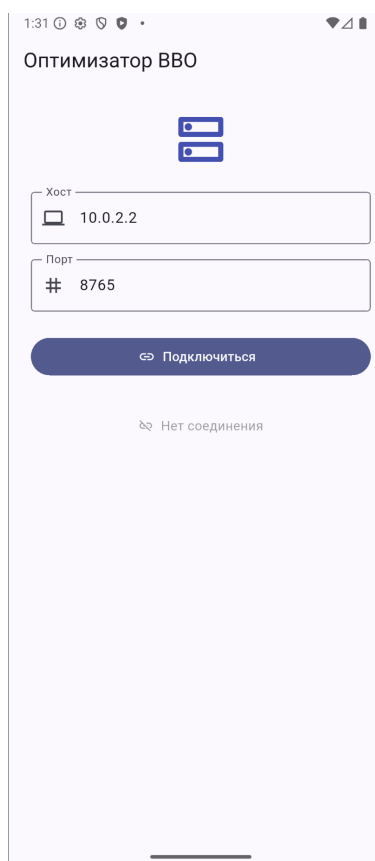
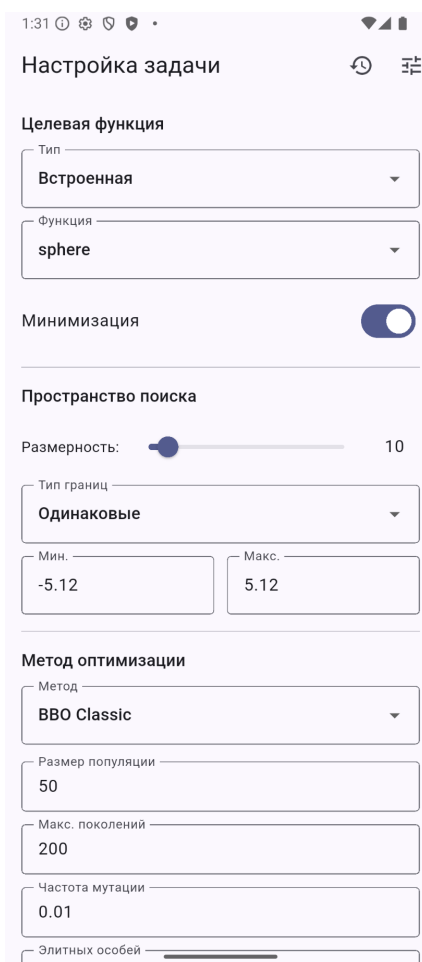


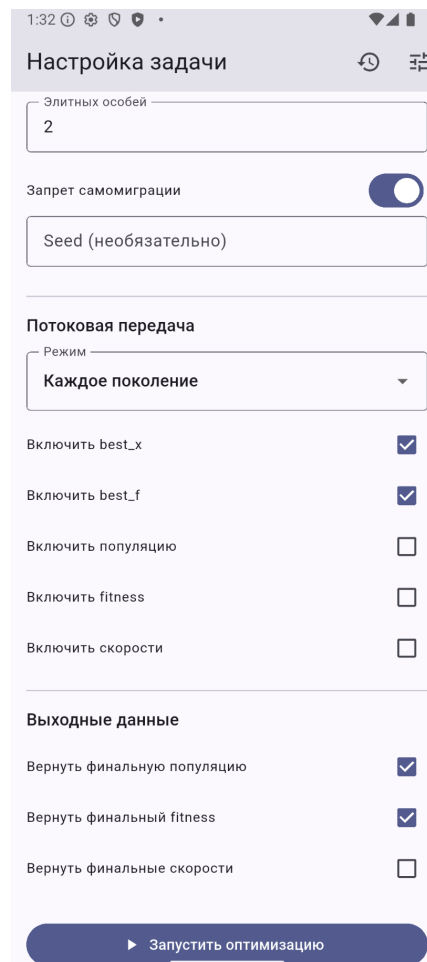
Рисунок 10 – Экран подключения к серверу

3.6.2 Настройка численного эксперимента

После подключения открывается основной экран настройки. Он разбит на блоки: целевая функция, постановка задачи, метод, режим потоковой передачи и состав выходных данных. В блоке функции выбирается встроенная функция или вводится выражение, а переключатель задаёт минимизацию или максимизацию. В блоке задачи ползунком задаётся размерность, ниже — границы области. В блоке метода выбирается алгоритм и его параметры: размер популяции, число поколений, вероятность мутации и другие. Настроек много, поэтому экран прокручивается; внизу — кнопка запуска. Вид экрана показан на рисунке 11.



а)



б)

Рисунок 11 – Экран настройки эксперимента:

а) – выбор функции и задачи;

б) – параметры метода и потоковой передачи

3.6.3 Выполнение и наблюдение за прогрессом

После запуска приложение переходит на экран выполнения. Вверху — индикатор прогресса с номером поколения и долей выполнения, ниже — лучшее значение и время. Основную часть занимает график сходимости, который достраивается в реальном времени. Внизу — кнопка отмены. Экран показан на рисунке 12.

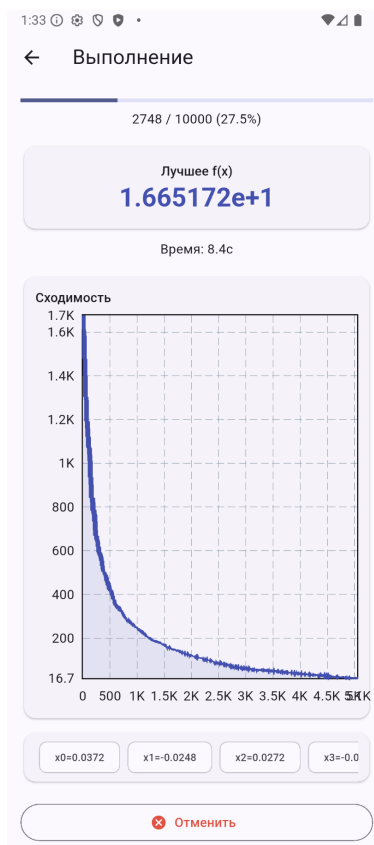
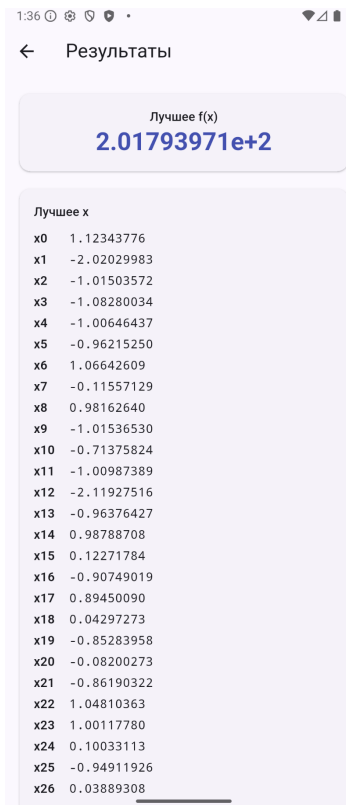


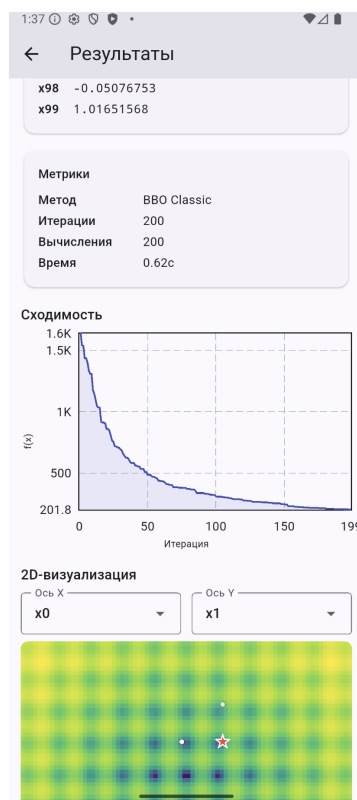
Рисунок 12 – Экран выполнения с графиком сходимости в реальном времени

3.6.4 Просмотр результата

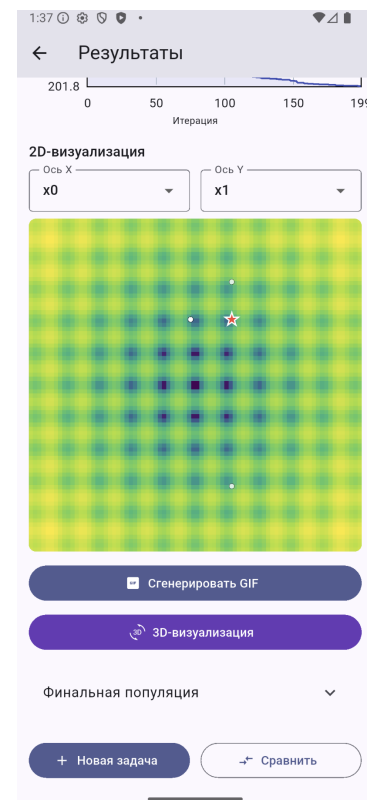
По завершении открывается экран результата. Вверху — лучшее значение функции, ниже координаты лучшего решения и сводка: метод, число поколений и вычислений функции, время. Далее идёт график сходимости за весь запуск, а под ним — контурная карта пространства поиска с лучшим решением и ближайшими особями. Здесь же кнопки анимации и перехода к трёхмерной визуализации. Вид экрана показан на рисунке 13.



а)



б)



в)

Рисунок 13 – Экран результата:
а) – лучшее решение;
б) – показатели и график сходимости;
в) – контурная карта пространства поиска

По нажатию кнопки строится анимация: контурная карта, на которой популяция смещается от поколения к поколению. Готовая анимация показывается прямо на экране, как на рисунке 14.

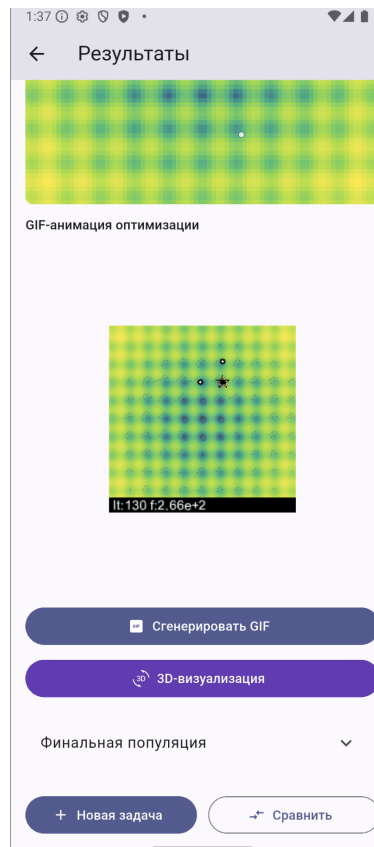


Рисунок 14 – Анимация процесса оптимизации на экране результата

3.6.5 Трёхмерная визуализация

Отдельный экран показывает функцию трёхмерной поверхностью. Высота точки — это значение функции, на поверхности отмечены лучшее решение и путь его улучшения. Поверхность вращается движением пальца, а выпадающие списки сверху задают, какие две координаты отложить по осям. Экран показан на рисунке 15.

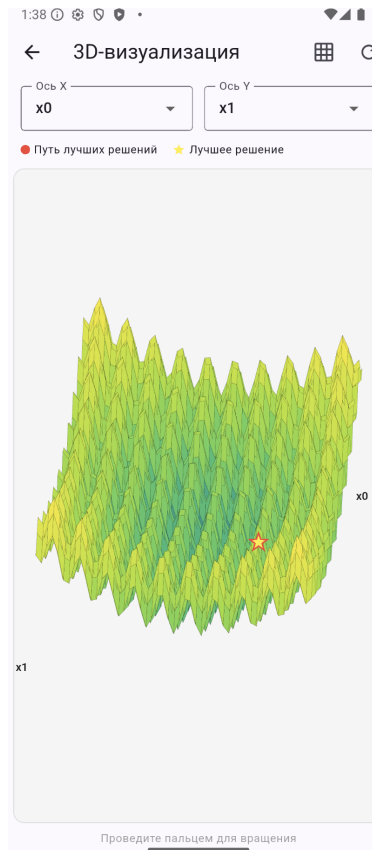


Рисунок 15 – Экран трёхмерной визуализации целевой функции

3.6.6 История и сравнение запусков

Все завершённые эксперименты попадают в историю. На её экране они показаны списком: метод, целевая функция, размерность, итоговое значение и время; список фильтруется по методу и функции. Отметив несколько запусков, пользователь переходит к сравнению. На экране сравнения — совмещённый график сходимости, таблица показателей и столбчатая диаграмма лучших значений. Оба экрана показаны на рисунке 16.



а)



б)

Рисунок 16 – Экраны истории и сравнения запусков:

а) – экран истории экспериментов;

б) – экран сравнения запусков

3.7 Реализация визуализации результатов

Визуализация выполнена на клиенте. График сходимости строится готовым средством, а контурная карта, поверхность и кадры анимации рисуются вручную через низкоуровневый холст. Ниже рассмотрен каждый вид.

3.7.1 График сходимости

График сходимости показывает, как убывает лучшее значение от поколения к поколению. Он строится готовым средством, которому передаётся список накопленных значений. На экране выполнения список пополняется по мере прихода прогресса, и график достраивается в реальном времени. Если значения убывают на несколько порядков, по оси ординат берётся логарифмическая шкала, иначе нижняя часть кривой сжимается у нуля. На экранах

результата и сравнения тот же график строится по полной истории, а при сравнении на одни оси ложится несколько кривых разного цвета.

3.7.2 Контурная карта

Контурная карта строится вручную. Область поиска разбивается на равномерную сетку, в каждой ячейке вычисляется значение функции, которое через логарифмическую нормировку переводится в цвет, и ячейка закрашивается прямоугольником. Нормировка делает различимыми и глубокие минимумы, и пологие участки. Поверх наносятся положения особей и отдельно лучшее решение. Основная часть рисования — в листинге 8.

Листинг 8: Построение контурной карты

```
1 for (int j = 0; j < resolution; j++) {
2   for (int i = 0; i < resolution; i++) {
3     final val = grid.values[j][i];
4     final t = range > 0
5       ? (math.log(val - minVal + 1) / math.log(range + 1)).clamp(0.0,
6         1.0)
7       : 0.0;
8     paint.color = ColorMaps.viridis(t);
9     canvas.drawRect(
10      Rect.fromLTWH(i * cellW, j * cellH, cellW + 0.5, cellH + 0.5),
11      paint,
12    );
13  }
```

Карта требует вычислить функцию во всех узлах, поэтому считается один раз при открытии экрана, а при перерисовке берутся готовые значения. При размерности больше двух по осям откладываются две выбранные координаты, остальные фиксируются — это даёт сечение функции в выбранной плоскости.

3.7.3 Трёхмерная поверхность

Трёхмерная поверхность строится по той же сетке. Узлы образуют четырёхугольные грани, которые проектируются на экран по формулам поворота: координаты узла нормируются к единому диапазону и поворачиваются вокруг вертикальной и горизонтальной осей, после чего две координаты дают точку на экране. Проекция показана в листинге 9.

Листинг 9: Проекция точки поверхности на экран

```

1 Offset project(double px, double py, double pz) {
2     final nx = 2 * (px - xMin) / (xMax - xMin) - 1;
3     final ny = 2 * (py - yMin) / (yMax - yMin) - 1;
4     final nz = 2 * (pz - fMin) / fRange - 1;
5
6     final rx = nx * cosZ - ny * sinZ;           // rotate around Z
7     final ry = nx * sinZ + ny * cosZ;
8     final ry2 = ry * cosX - nz * sinX;         // rotate around X
9
10    return Offset(cx + rx * scale, cy - ry2 * scale);
11 }

```

Чтобы ближние грани перекрывали дальние, применяется алгоритм ху-дожника: грани сортируются по глубине от дальних к ближним и рисуются в этом порядке. Цвет грани задаётся средним значением функции в её узлах по той же шкале, что и карта. Поверх наносится путь лучшего решения — последовательность точек за время поиска; чтобы не рисовать тысячи точек, путь прореживается. Углы поворота меняются жестом, и при каждом изменении поверхность проектируется заново.

3.7.4 Анимация процесса оптимизации

Анимация показывает, как популяция смещалась от поколения к поколению. Она собирается кадром в формат GIF. Сначала строится фоновый кадр с контурной картой, затем для каждого кадра поверх фона наносятся особи, лучшее решение и строка с номером поколения и лучшим значением. Число кадров фиксировано, поэтому из истории берётся равномерная выборка поколений. Кадры собираются в файл с заданной частотой смены, причём последний показывается дольше. Сборка кадров — в листинге 10.

Листинг 10: Покадровая сборка анимации

```

1 final step = math.max(1, totalIterations ~/ frameCount);
2 for (int f = 0; f < frameCount; f++) {
3     final iterIndex = math.min(f * step, totalIterations - 1);
4     final progress = (f + 1) / frameCount;
5     final frame = img.Image.from(bgFrame);
6
7     final visibleCount =
8         (population.length * progress).round().clamp(1, population.length);
9
10    for (int p = 0; p < visibleCount; p++) {
11        final px = _mapCoord(population[p][dimX], xMin, xMax, size);
12        final py = _mapCoord(population[p][dimY], yMin, yMax, size);

```

```
12     _drawDot(frame, px, py, 2, img.ColorRgba8(255, 255, 255, 200));  
13 }  
14 _drawStar(frame, bx, by, starSize, img.ColorRgba8(255, 0, 0, 255));  
15 _drawInfoBar(frame, iterIndex, historyBestF[iterIndex], size);  
16 frames.add(frame);  
17 }
```

Готовая анимация показывается прямо на экране результата и может быть сохранена как файл. Формат GIF не требует внешних средств воспроизведения и собирается из готовых кадров, поэтому удобен здесь.

4 Аналитическая часть

4.1 Тестирование программного обеспечения

Правильность работы комплекса проверялась двумя способами. Серверная часть с вычислительным ядром и логикой обработки задач покрыта автоматическими тестами. Клиентская часть, в основе которой интерфейс, проверялась вручную по сценариям. Логiku сервера удобно проверять программно, а работу интерфейса — наблюдением на устройстве.

4.1.1 Автоматическое тестирование серверной части

Серверная часть протестирована библиотекой `pytest` [30]. Написано 73 теста по группам модулей; они охватывают и отдельные функции, и их взаимодействие:

1. Тесты очереди задач. Проверялись постановка с учётом приоритетов, извлечение в правильном порядке, отмена ожидающих и выполняемых задач, защита от повторов и переполнения. 14 тестов.
2. Тесты целевых функций. Проверялись вычисление встроенных функций в известных точках, разбор и вычисление пользовательских выражений, отклонение небезопасных и некорректных. 15 тестов.
3. Тесты протокола обмена. Проверялись построение и разбор конверта, сохранение полей заголовка и обработка всех типов сообщений. 11 тестов.
4. Тесты реестра методов. Проверялись регистрация всех семи методов, создание алгоритма по имени и ошибка при неизвестном методе. 4 теста.
5. Тесты модуля запуска. Проверялся полный прогон: запуск алгоритма, передача прогресса, прерывание по отмене и подготовка результата к передаче. 7 тестов.
6. Тесты валидации. Проверялось, что корректные параметры проходят, а некорректные — неизвестный метод, неположительная размерность, несогласованные границы — отклоняются. 14 тестов.
7. Тесты соединения. Проверялась работа обработчика отдельно и в связке с очередью: рукопожатие, маршрутизация, постановка задачи и ответы. 8 тестов.

Для примера в листинге 11 приведён тест, проверяющий, что в реестре зарегистрированы все семь методов.

Листинг 11: Пример теста: проверка состава реестра методов

```
1 def test_all_methods_registered(self):
2     expected = {
3         "bbo.classic", "bbo.classic.modified",
4         "bbo.sinusoidal", "bbo.sinusoidal.modified",
5         "bbo.mixed", "bbo.mixed.modified",
6         "pso",
7     }
8     assert set(METHODS.keys()) == expected
```

Все 73 теста проходят успешно. Это подтверждает, что вычислительное ядро, очередь, протокол и логика обработки задач работают как задумано, а изменения в коде не ломают уже проверенное поведение.

4.1.2 Функциональное тестирование клиента

Клиентская часть проверялась вручную: на устройстве последовательно выполнялись основные сценарии использования, и поведение приложения сравнивалось с ожидаемым. Тестирование охватывало все экраны и основные действия пользователя.

1. Подключение к серверу. Приложение устанавливает соединение по адресу и порту, показывает его состояние и восстанавливает при кратком обрыве. При неверном адресе сообщает об ошибке и не зависает.
2. Настройка и запуск задачи. На экране настройки можно выбрать метод и функцию, задать размерность, границы и параметры, ввести своё выражение. После запуска открывается экран выполнения, а недопустимые значения не дают начать вычисление.
3. Наблюдение за прогрессом. Во время вычисления видны номер поколения и лучшее значение, график достраивается в реальном времени. Кнопка отмены прерывает вычисление, и приложение корректно обрабатывает это.
4. Просмотр результата. После завершения видны лучшее решение, показатели и график; строятся контурная карта, поверхность и анимация. Поверхность вращается жестом, а выбор координат меняет сечение.
5. Работа с историей. Эксперименты сохраняются, список фильтруется по методу и функции, любую запись можно открыть заново. История сохраняется между запусками.

6. Сравнение запусков. Несколько выбранных запусков выводятся вместе на графике, в таблице и на диаграмме, и сравнение остаётся читаемым при разных методах и функциях.

Все сценарии отработали как ожидалось. Вместе с автоматическими тестами сервера это позволяет считать комплекс работоспособным и готовым к экспериментам.

4.2 Вычислительные эксперименты и анализ результатов

Главная цель экспериментов — сравнить методы между собой и оценить влияние модификаций базового алгоритма. Для этого все семь методов запускались на наборе тестовых функций при разных размерностях.

4.2.1 Методика экспериментов

Использовались четыре функции из подраздела 1.1: сфера, Растригина, Экли и Букина N.6. Сфера имеет единственный минимум и проверяет скорость сходимости. Растригина и Экли содержат множество локальных минимумов и проверяют, не застревает ли метод. Букина N.6 определена только для двух переменных. Сфера, Растригина и Экли запускались в размерностях 2, 10, 30 и 50; вместе с двумерной Букина N.6 это тринадцать задач.

На каждой задаче метод запускался несколько раз с разными seed, а результаты усреднялись. Параметры подбирались предварительно. Качество оценивалось средним и медианным лучшими значениями, их разбросом, наилучшим значением по всем запускам и средним временем счёта. Минимум всех функций равен нулю, поэтому меньшее значение означает более точный результат.

4.2.2 Сравнение методов на функции с множеством минимумов

Наиболее показательна функция Растригина: у неё много локальных минимумов, и разница между методами хорошо видна. Средние лучшие значения при разных размерностях приведены в таблице 1.

Таблица 1 – Среднее лучшее значение на функции Растригина

d	ВВО кл.	ВВО кл. мод.	ВВО син. мод.	ВВО см. мод.	PSO
2	0,159	0,019	0,007	0,010	0
10	1,47	0,67	0,65	0,91	5,72
30	14,40	8,08	8,47	8,27	42,50
50	36,0	30,1	28,5	25,6	95,0

При размерности 2 лучший результат у PSO — он находит минимум точно. С ростом размерности картина меняется: начиная с десяти переменных все варианты ВВО заметно опережают PSO, и отрыв растёт. При пятидесяти переменных лучшие варианты ВВО дают около 25–30, а PSO — около 95. Причина в устройстве методов: рой частиц быстро стягивается к одной точке и застревает в одном из минимумов, тогда как покомпонентный обмен в ВВО дольше хранит разнообразие популяции. Та же закономерность показана на рисунке 17.

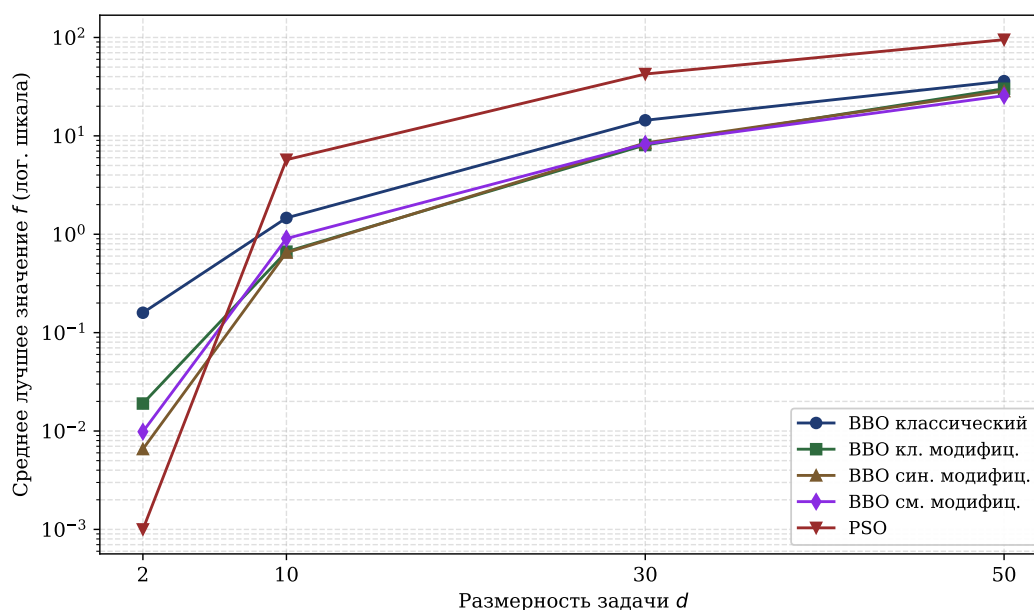


Рисунок 17 – Зависимость среднего лучшего значения на функции Растригина от размерности задачи

Сравнение базовых и модифицированных вариантов показывает, что модификации почти всегда улучшают результат. На Растригина любой модифицированный вариант точнее классического во всех размерностях, а в среднем по всем тринадцати задачам модификация не хуже базового вари-

анта в двенадцати-тринадцати случаях из тринадцати. Значит, адаптивная мутация, рестарт и локальный поиск действительно помогают.

4.2.3 Сравнение методов на разных функциях

Для общей картины в таблице 2 приведены средние лучшие значения на трёх функциях при размерности 30.

Таблица 2 – Среднее лучшее значение на разных функциях при $d = 30$

Метод	Сфера	Растригина	Экли
ВВО классический	0,23	14,40	3,69
ВВО кл. модифиц.	0,18	8,08	3,35
ВВО син. модифиц.	0,13	8,47	2,67
ВВО см. модифиц.	0,15	8,27	3,32
PSO	0	42,50	0,29

Таблица подтверждает, что у методов разные сильные стороны. На сфере и Экли с их более гладким рельефом PSO заметно точнее: на сфере — на несколько порядков. А на Растригина PSO, наоборот, уступает ВВО в несколько раз. Среди вариантов ВВО различия между моделями миграции невелики — значителен вклад модификаций, а не моделей миграции.

За точность приходится платить временем. На Растригина при размерности 30 один запуск классического ВВО занимает около 0,65 секунды, а модифицированного — около 5 секунд: основное время уходит на локальный поиск. PSO самый быстрый — около 0,02 секунды, ведь его итерация — это простые операции над массивами. Значит, выбор метода — это выбор между скоростью и точностью на сложном рельефе.

4.2.4 Выводы по результатам

PSO предпочтителен на задачах с простым рельефом и невысокой размерностью, где важна скорость. На задачах с множеством локальных минимумов и высокой размерностью лучше работают варианты ВВО, причём влияют механизмы адаптации: адаптивная мутация, рестарт и локальный поиск стабильно улучшают результат. Различия между моделями миграции невелики. Эти результаты согласуются с теоретическими представлениями о поведении методов.

ЗАКЛЮЧЕНИЕ

В ходе работы разработан программный комплекс для исследования и визуализации модификаций алгоритма биогеографической оптимизации.

Реализовано серверное вычислительное ядро из семи методов: классический ВВО, его варианты с синусоидальной и смешанной миграцией, три их модификации с адаптивной мутацией, рестартом и локальным поиском, а также метод роя частиц. Алгоритмы запускаются на четырёх встроенных функциях и на функции, заданной выражением; выражение перед вычислением проверяется на безопасность.

Разработан протокол поверх WebSocket. Он поддерживает постановку задачи, передачу прогресса в реальном времени, выдачу результата и отмену, а очередь с приоритетами и пул обработчиков позволяют обслуживать несколько клиентов сразу.

Создано мобильное приложение: через него настраивается эксперимент, отправляется задача и проводится наблюдение за вычислением. Оно построено по слоям, а связь с сервером и хранение истории вынесены в отдельные слои.

Реализованы средства визуализации: график сходимости в реальном времени, контурная карта и трёхмерная поверхность с вращением, а также анимация популяции. Это позволяет увидеть как итог, так и сам процесс поиска.

Организовано хранение истории на устройстве и сравнение запусков. Сохранённые результаты сопоставляются на совмещённом графике, в таблице и на диаграмме — в этом и состоит назначение комплекса.

Также были проведены эксперименты: семь методов сравнивались на тринадцати задачах разной структуры и размерности. Рой частиц точнее на простом рельефе и низкой размерности, а на функциях с множеством минимумов и высокой размерностью выигрывают варианты ВВО. Механизмы адаптации почти всегда улучшают базовый алгоритм ценой времени, а различия между моделями миграции невелики.

Комплекс допускает дальнейшее развитие. Семейство методов расширяется другими метаэвристиками через реестр, без изменения остального кода. Набор функций можно дополнить, а заложенные в протокол разбиение сообщений и сжатие — задействовать при больших объёмах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Карпенко А. П.* Современные алгоритмы поисковой оптимизации. Алгоритмы, вдохновлённые природой : учебное пособие. — 3-е изд. — Москва : Изд-во МГТУ им. Н. Э. Баумана, 2021. — 446 с.
2. *Гладков Л. А., Курейчик В. В., Курейчик В. М.* Генетические алгоритмы. — Москва : Физматлит, 2010. — 368 с.
3. *Пантелеев А. В., Метлицкая Д. В., Алешина Е. А.* Методы глобальной оптимизации. Метаэвристические стратегии и алгоритмы. — Москва : Вузовская книга, 2013. — С. 244.
4. *Саймон Д.* Алгоритмы эволюционной оптимизации / пер. А. В. Логунов. — Москва : ДМК Пресс, 2020. — 1002 с.
5. *Карпенко А. П.* Эволюционные операторы популяционных алгоритмов глобальной оптимизации. Опыт систематизации // Математика и математическое моделирование. — 2018. — № 1. — С. 59—89.
6. *Родзин С. И., Скобцов Ю. А., Эль-Хатиб С. А.* Биоэвристики: теория, алгоритмы и приложения. — Чебоксары : Изд-во «Среда», 2019. — 224 с.
7. *Kennedy J., Eberhart R.* Particle swarm optimization // Proceedings of ICNN'95 — International Conference on Neural Networks. Т. 4. — 1995. — С. 1942—1948.
8. *Карпенко А. П., Щербакова Н. О., Буланов В. А.* Гибридный алгоритм глобальной оптимизации на основе алгоритмов искусственной иммунной системы и метода роя частиц // Наука и образование (электрон. журн. МГТУ им. Н. Э. Баумана). — 2014. — № 3. — С. 255—271.
9. *Казакова Е. М.* Краткий обзор методов оптимизации на основе роя частиц // Вестник КРАУНЦ. Физико-математические науки. — 2022. — Т. 19, № 2. — С. 150—174.
10. *Карпенко А. П., Синяговская О. А.* Глобальная оптимизация методом биогеографии // Наука и образование (электрон. журн. МГТУ им. Н. Э. Баумана). — 2013. — № 10. — С. 373—398.

11. К вопросу эффективности методов и алгоритмов решения оптимизационных задач с учётом специфики целевой функции / Е. Н. Остроух [и др.] // Вестник Донского государственного технического университета. — 2019. — Т. 19, № 1. — С. 81—85.
12. Новикова Е. С., Бекенева Я. А., Бестужев М. П. Методы визуального анализа для мониторинга данных от сенсорных сетей // Известия СПбГ-ЭТУ «ЛЭТИ». — 2020. — № 8/9. — С. 52—60.
13. CompuServe Incorporated. GIF89a Graphics Interchange Format Version 89a. — 1990. — 34 с.
14. Chacón Sartori C., Blum C., Ochoa G. STNWeb: A new visualization tool for analyzing optimization algorithms // Software Impacts. — 2023. — Т. 17. — С. 100558.
15. Acerbi L. OptimViz: Visualize optimization algorithms in MATLAB. — URL: <https://github.com/lacerbi/optimviz> (дата обр. 21.05.2026).
16. Dupont E. Interactive Visualization of Optimization Algorithms in Deep Learning. — URL: <https://emiliendupont.github.io/2018/01/24/optimization-visualization/> (дата обр. 21.05.2026).
17. Schäpermeier L., Grimme C., Kerschke P. To Boldly Show What No One Has Seen Before: A Dashboard for Visualizing Multi-objective Landscapes // Evolutionary Multi-Criterion Optimization. Lecture Notes in Computer Science. — 2021. — Т. 12654. — С. 632—644.
18. RFC 6455: The WebSocket Protocol : RFC Editor. — 2011. — URL: <https://www.rfc-editor.org/rfc/rfc6455> (дата обр. 22.05.2026).
19. RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format : RFC Editor. — 2017. — URL: <https://www.rfc-editor.org/rfc/rfc8259> (дата обр. 22.05.2026).
20. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы. — 5-е изд. — Санкт-Петербург : Питер, 2016. — 992 с.
21. Федюлов А. А. Проектирование архитектуры протокола клиент-серверного взаимодействия web-приложений на основе websocket // Программные системы и вычислительные методы. — 2025. — № 3. — С. 20—30.

22. NumPy documentation : NumPy documentation. — URL: <https://numpy.org/doc/stable/> (дата обр. 22.05.2026).
23. *Яворски М., Зуаде Т.* Python. Лучшие практики и инструменты. — Санкт-Петербург : Питер, 2024. — 592 с.
24. FastAPI : FastAPI documentation. — URL: <https://fastapi.tiangolo.com/> (дата обр. 22.05.2026).
25. json — JSON encoder and decoder : Python 3 documentation. — URL: <https://docs.python.org/3/library/json.html> (дата обр. 09.04.2026).
26. dart:convert : Dart documentation. — URL: <https://dart.dev/libraries/dart-convert> (дата обр. 09.04.2026).
27. Сравнительный анализ форматов сериализации и передачи данных JSON, XML, CBOR и GPB / В. Д. Шульман [и др.] // StudNet. — 2021. — Т. 4, № 7. — С. 1686—1696.
28. *Рябоконь О. С., Кукарцев В. В.* Новый язык структурного веб-программирования Dart // Актуальные проблемы авиации и космонавтики. — 2013. — Т. 1, № 9. — С. 436—437.
29. *Камышев А. О., Зотин А. Г.* Особенности использования фреймворка Flutter при разработке кроссплатформенных приложений // Актуальные проблемы авиации и космонавтики: сб. материалов VI Междунар. науч.-практ. конф. Т. 2. — Красноярск : СибГУ им. М. Ф. Решетнёва, 2020. — С. 138—140.
30. pytest: helps you write better programs : pytest documentation. — URL: <https://docs.pytest.org/en/stable/> (дата обр. 22.05.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Руководство пользователя

Руководство описывает работу с мобильным приложением комплекса — от запуска сервера до анализа результатов. Предполагается, что серверная часть уже запущена и доступна по сети, а мобильное приложение установлено на устройство.

Перед началом работы необходимо запустить сервер на компьютере, доступном по сети. Сервер начинает принимать подключения по указанному адресу и порту. После этого работа ведётся через мобильное приложение в следующем порядке.

1. Подключение к серверу. При запуске приложение открывает экран подключения. В поля вводятся адрес и порт сервера, после чего нажимается кнопка подключения. Если соединение установлено, приложение переходит к экрану настройки; иначе под кнопкой выводится сообщение об ошибке, и адрес следует проверить. Внешний вид экрана приведён на рисунке 10.
2. Настройка эксперимента. На экране настройки выбирается целевая функция — встроенная или заданная собственным выражением, — задаются размерность и границы области поиска, выбирается метод оптимизации и его параметры. Для большинства параметров уже подставлены разумные значения по умолчанию, поэтому при первом знакомстве достаточно выбрать метод и функцию. Экран настройки показан на рисунке 11.
3. Запуск и наблюдение за вычислением. После нажатия кнопки запуска приложение переходит к экрану выполнения, где отображаются номер текущего поколения, лучшее найденное значение и график сходимости, обновляемый в реальном времени. При необходимости вычисление можно прервать кнопкой отмены. Этот экран показан на рисунке 12.

4. Просмотр результата. По завершении вычисления открывается экран результата с лучшим решением, показателями качества, графиком сходимости и контурной картой пространства поиска. Здесь же доступны построение анимации и переход к трёхмерной визуализации, которую можно вращать движением пальца. Вид экрана результата приведён на рисунках 13–15.
5. Работа с историей и сравнение. Каждый завершённый эксперимент сохраняется в историю, доступную с экрана настройки. В истории запуски можно фильтровать, открывать повторно и отбирать несколько штук для сравнения. На экране сравнения выбранные запуски выводятся вместе — на совмещённом графике сходимости, в таблице показателей и на столбчатой диаграмме. Экраны истории и сравнения показаны на рисунке 16.

При потере соединения с сервером приложение пытается восстановить его автоматически. Если сервер недоступен длительное время, следует проверить, запущен ли он, и повторить подключение вручную с экрана подключения.